FRAUDLENS

A Real-Time, Explainable Fraud Detection Platform with an Agentic AI Investigation Copilot

---

Complete Technical Project Documentation & Learning Handbook (Omnibus Edition)

Author: Dhanraj Shitole

Degree: Bachelor of Technology (BTech) — Final-Year Capstone Project

Academic Session: 2025 – 2026

Repository: https://github.com/DhanrajShitole/fraudlens

Core Technology Stack:

Python 3.13 • LightGBM • SHAP TreeExplainer • SMOTE • Ollama Llama 3.1 8B
FastAPI • React Dashboard • SQLite • MLflow Registry • PSI Drift • Docker

EXECUTIVE NOTICE & SCOPE OF THIS COMPREHENSIVE COMPILATION:

This publication-grade handbook represents the definitive single source of truth for the FraudLens platform. It consolidates all technical guides, architectural schematics, mathematical formulations, evidence-grounded agent prompt contracts, anti-hallucination validation proofs, REST API references, DevOps containerization workflows, defense viva questions, and self-assessment checklists.

Handbook Table of Contents

PART I: EXECUTIVE OVERVIEW & SYSTEM DESIGN

1. Executive Summary	*Problem statement, core architectural innovations, and verified metrics*
2. Technical Architecture Deep Dive	*Offline vs online topology, sequence diagrams, and schema specifications*

PART II: MACHINE LEARNING & EXPLAINABLE AI

3. Machine Learning Deep Dive	*Mathematical formulations: LightGBM objective, SMOTE synthesis, TreeSHAP*
4. AI Agent & LLM Copilot Deep Dive	*Deterministic evidence tools, TF-IDF policy engine, prompt template, validator*

PART III: SYSTEM INTEGRATION & MLOPS

5. API & Integration Reference	*FastAPI architecture, lifespan loader, 5 endpoints with JSON schemas*
6. DevOps & MLOps Guide	*MLflow registry with 'production' alias, PSI drift detection, Docker config*
7. Setup & Troubleshooting Guide	*Cold-start reproduction, environment setup, CLI triggers, troubleshooting*

PART IV: ACADEMIC DEFENSE & VIVA PREPARATION

8. Presentation, Demo & Viva Guide	*Slide-by-slide structure, live demonstration script, 50+ viva Q&As;*

PART V: MASTER 20-CHAPTER LEARNING HANDBOOK

Chapters 1 – 20 Comprehensive	*Full textbook-grade chapters, file walkthroughs, trade-offs, glossary & checklist*

PART I: EXECUTIVE SUMMARY

FraudLens — Executive Summary & Project Brief

Project Name: FraudLens

Subtitle: A Real-Time, Explainable Financial Fraud Detection Platform with an Evidence-Grounded Agentic Investigation Copilot

Author: Dhanraj Shitole

Status: Completed & Validated (Phases 1–12 Complete)

Repository: https://github.com/DhanrajShitole/fraudlens

---

1. Problem Statement & Motivation

Financial institutions process billions of transactions daily, but fraudulent events constitute a tiny fraction of total volume (typically 0.1% to 3.5%). This extreme **class imbalance** makes accurate automated detection difficult. More critically, the operational bottleneck in modern anti-fraud operations has shifted from *detection* to *investigation throughput*:

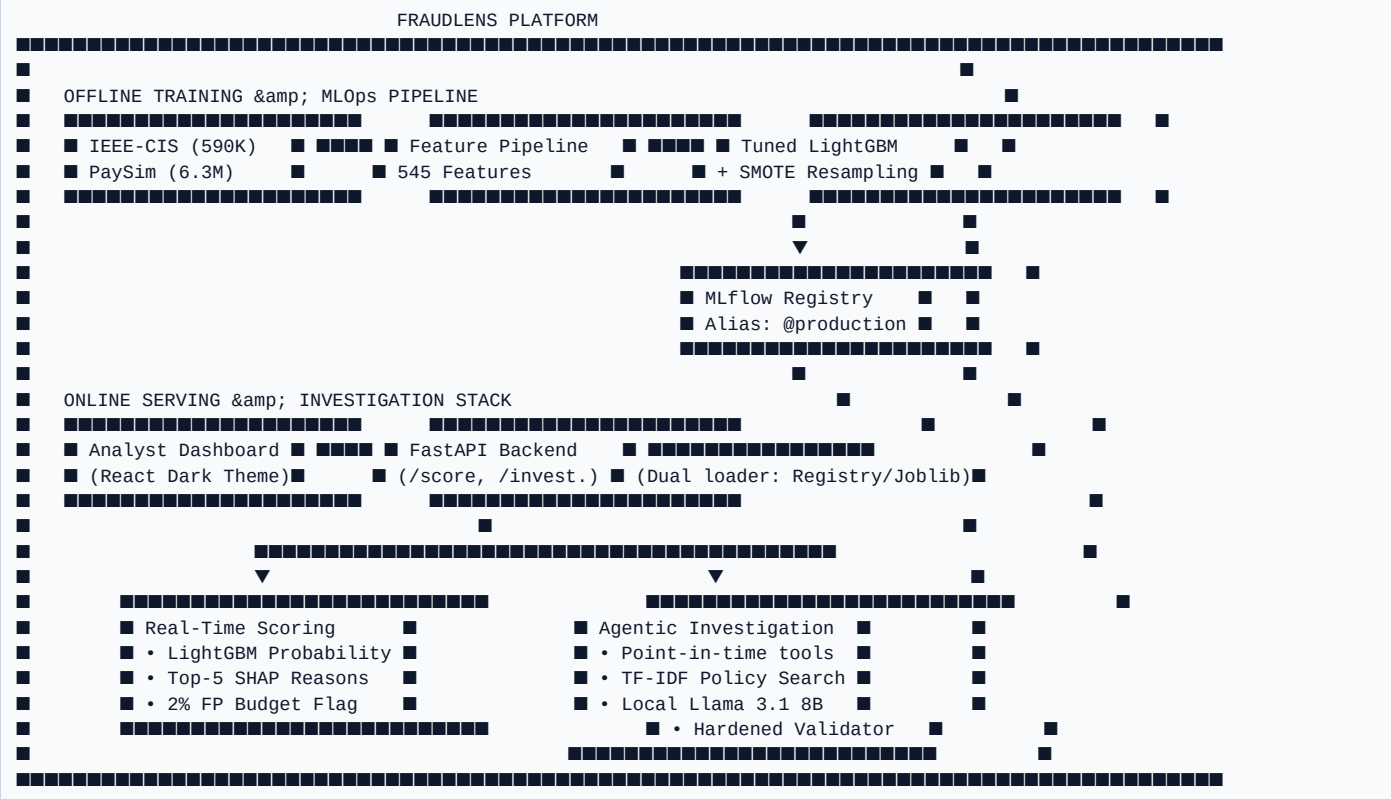
- When a machine learning model flags an alert, a human compliance analyst must manually cross-reference customer profiles, recent card velocity, IP/device associations, and regulatory compliance rules.
- Assembling this contextual evidence takes 15 to 45 minutes per suspicious transaction.
- Traditional rules-based engines produce high false-positive rates, inundating analysts with alert fatigue.
- Generic Generative AI / Large Language Models (LLMs) cannot be deployed naively in compliance workflows because unconstrained models hallucinate facts, invent geographic claims, leak future labels, and contradict their own data.

FraudLens was conceived and engineered to solve both sides of this equation:

1. **Accurate, Cost-Sensitive Fraud Scoring:** Gradient-boosted decision trees (LightGBM) optimized under strict false-positive budget constraints (2% false-positive budget) paired with TreeSHAP explainability to provide instant mathematical reason codes.
2. **Grounded Agentic Copilot:** A locally hosted, tool-scoped AI agent (Llama 3.1 8B via Ollama) governed by the core architectural rule: *"Code makes decisions, language models write prose."* The copilot deterministically extracts point-in-time entity evidence and organizational policy citations, while the LLM drafts structured narrative summaries that are strictly validated against tool evidence before reaching the human investigator.

---

2. High-Level System Overview



---

3. Key Achievements & Verified Performance Metrics

All metrics reported below are extracted directly from the frozen, held-out test evaluation reports (`reports/test_evaluation.md`) and validation experiments (`reports/`):

A. Machine Learning Scoring Performance (IEEE-CIS Benchmark)

- **Validation PR-AUC: 0.5975** (Tuned LightGBM + SMOTE) — a **+7.3% relative improvement** over the untuned baseline (0.5566).
- **Validation ROC-AUC: 0.9242** (vs Logistic Regression 0.8180).
- **Validation F1 @ 2% False-Positive Budget: 0.5405** (Precision: 50.71%, Recall: 57.86% at fixed threshold $\tau = 0.129365$).
- **Held-Out Test Set Generalization (One-Shot):**
- **Test PR-AUC: 0.5324** — delivering a **15.30x lift** over the test set base fraud rate (3.48%).
- **Test ROC-AUC: 0.9066**.
- **Test F1 @ Fixed Threshold: 0.4821** (Recall: 53.39%, Precision: 43.95%, Actual False-Positive Rate: 2.46%).
- **Probability Calibration:** Test Brier score of **0.02213** vs. non-informative baseline of **0.03359**, yielding a **Brier Skill Score of 34.12%**.

B. The Anomaly Detection Hypothesis Test (PaySim Benchmark)

- Benchmarked supervised gradient boosting against unsupervised anomaly detection (Isolation Forest) across 6.36 million transactions (1,221.6:1 class imbalance).
- **Finding:** Supervised LightGBM dramatically outperformed Isolation Forest (ROC-AUC **0.949** vs. **0.840**; Recall at 2% FP budget **94.5%** vs. **2.5%**).
- **Conclusion:** Synthetic mobile money fraud follows crisp, discrete account-draining logic that supervised trees learn efficiently, while Isolation Forest flags high-value benign transfers.

C. Game-Theoretic Feature Attribution (TreeSHAP)

- Engineered point-in-time entity features grouping physical card and billing region `(card1, addr1)` proved overwhelmingly significant:
- `card1_addr1_amt_mean`: Ranked **13 / 545** (Top 2.4% most influential features).
- `card1_addr1_count`: Ranked **17 / 545** (Top 3.1%).
- `card1_addr1_amt_std`: Ranked **24 / 545** (Top 4.4%).
- `card1_addr1_email_count`: Ranked **39 / 545** (Top 7.2%).

D. Grounded Agent Reliability

- Solved 4 major LLM failure modes (geographic hallucination, data granularity errors, temporal label leakage, and vacuous comparisons).
- Tightened automated validator (`_model_text_clean`): Rejects ungrounded numbers, numbers sourced from policy text or notes, and comparative framing lacking two distinct grounded figures.
- Seamless fallback: If the local LLM crashes or fails validation after one retry, the agent generates a complete, deterministic, evidence-backed case file with 0% data loss.

---

4. Technology Stack & Operational Architecture

Layer	Selected Technology	Technical Justification
Language & Runtime	Python 3.13	Pinned, modern runtime with native typing and performance improvements.
Data Processing	Pandas 3.0, NumPy 2.2, PyArrow 25.0	High-performance Parquet I/O, vectorized split filtering, zero memory copies.
Predictive ML	LightGBM 4.7, Scikit-Learn 1.8, Imbalanced-Learn 0.14	Gradient-boosted decision trees with histogram binning and SMOTE resampling.
Explainability	SHAP 0.52 (TreeExplainer)	Exact Shapley value calculation in polynomial time for tree ensembles.
Local LLM Engine	Ollama 0.6.2, Llama 3.1 8B	Local, offline inference ensuring 100% data privacy (zero PII sent to cloud APIs).
Serving Backend	FastAPI 0.136, Uvicorn 0.46, Pydantic 2.13	Async REST endpoints, sub-15ms scoring latency, strict schema validation.
Case Store	SQLite 3 (`cases.db`)	Zero-configuration, serverless relational persistence for investigation audit trails.
Analyst UI	React 18 (via CDN), CSS3 Variables	Single-file analyst dashboard, dark theme, zero npm build dependencies.
MLOps & Monitoring	MLflow 3.16, direct PSI computation	Experiment tracking, model registry with `@production` alias, Population Stability Index drift tracking.
DevOps & CI/CD	Docker, Docker Compose, GitHub Actions	Pinned multi-stage container build and automated CI running 51 unit/integration tests.

---

5. Quality Assurance & Test Verification

FraudLens maintains a comprehensive test suite of **51 automated tests**, achieving 100% pass rates in local and CI environments:

```
python -m pytest tests/ -v
# Result: 51 passed, 2 warnings in ~10.7 seconds
```

- `tests/test_agent_logic.py` **(19 tests):** Validates point-in-time isolation, label leakage protection, numerical grounding, comparative framing rejection, policy matching, and Ollama failure resilience.

- **`tests/test_api.py` (12 tests):** Validates `/health`, `/score`, `/investigate`, `/cases`, HTTP status codes (200, 404, 503), and SQLite persistence.
- **`tests/test_drift.py` (6 tests):** Validates Population Stability Index calculation across stable distributions, large shifts, near-constant features, and missing values.
- **`tests/test_model.py` (14 tests):** Validates model file integrity, 545-feature schema alignment, probability distribution axioms, and TreeSHAP top-5 indexing.

---

6. How FraudLens Is Used

1. Scoring a Transaction:

- The compliance analyst inputs a Transaction ID into the dashboard.
- The backend runs LightGBM inference and computes TreeSHAP reason codes.
- The dashboard displays the calibrated risk score, an alert badge (**`FLAGGED FOR REVIEW`** vs **`BELOW THRESHOLD`** based on the 2% false-positive budget threshold $\tau = 0.129365$), and horizontal bars indicating which features increased or decreased risk.

2. Triggering an AI Investigation:

- The analyst clicks **Investigate**.
- The backend gathers historical entity transactions (point-in-time, up to this transaction only), calculates distinct email clusters on the same card/address, and matches organizational fraud policies via TF-IDF with a relevance floor.
- Local Llama 3.1 8B synthesizes the entity overview and bulleted risk factors.
- The output is validated: if clean, it is merged with code-generated policies and recommended actions; if ungrounded, the system transparently falls back to deterministic evidence bullets.
- The full case file is rendered in the dashboard and permanently logged to SQLite.

PART II: TECHNICAL ARCHITECTURE DEEP DIVE

FraudLens — Technical Architecture & Component Interaction Specification

Document Version: 1.0

Author: Dhanraj Shitole

Target Audience: Software Architects, AI/ML Engineers, Technical Examiners

---

1. Architectural Philosophy: The Decoupled Hybrid Pattern

Production financial compliance platforms face two conflicting requirements:

1. **Low Latency & High Mathematical Rigor:** Scoring transactions must occur in milliseconds, with fully deterministic, auditable probability calculations and exact feature attributions.
2. **Context-Rich Narrative Synthesis:** Compliance officers need readable case summaries cross-referencing multi-table histories and company policy guidelines.

Attempting to solve both with an end-to-end Large Language Model fails because generative models are inherently non-deterministic, computationally expensive, and susceptible to hallucination. Attempting to solve both with traditional rule engines fails because complex fraud rings evade static thresholds and rules lack narrative flexibility.

FraudLens resolves this tension through a Decoupled Hybrid Pattern:

- **Code makes decisions; models score; LLMs narrate.**
- Machine learning (LightGBM) outputs the numerical fraud probability and flags alerts against an empirical false-positive budget.
- SHAP (TreeExplainer) deterministically decomposes predictions into exact feature contributions.
- Deterministic Python tools query point-in-time entity histories and execute vector-space policy matching with strict relevance floors.
- The Local LLM (Llama 3.1 8B via Ollama) is constrained to writing exactly two sentences (`ENTITY SUMMARY` and `TOP RISK FACTORS`).
- An automated validator inspects all generated text; any failure drops directly to a deterministic, code-assembled case file.

---

2. Global System Architecture

```
graph TD
    subgraph Data Layer
        D1[IEEE-CIS Dataset 590K] --&gt; DP[src/features.py]
        D2[PaySim Dataset 6.3M] --&gt; DP
        DP --&gt; P1[data/processed/ieee_train.parquet]
        DP --&gt; P2[data/processed/ieee_val.parquet]
        DP --&gt; P3[data/processed/ieee_test.parquet]
    end

    subgraph Offline MLOps Pipeline
        P1 --&gt; TR[src/train_advanced.py]
        P2 --&gt; TR
        TR --&gt; M1[models/ieee_lightgbm_tuned.joblib]
        TR --&gt; M2[models/ieee_lightgbm_tuned_metadata.json]
        TR --&gt; MLF[(MLflow SQLite: mlflow.db)]
        M1 --&gt; REG[src/register_model.py]
        REG --&gt; REG_ALIAS[Model Registry: @production]
        P1 --&gt; DRIFT[src/drift_check.py]
        P2 --&gt; DRIFT
        DRIFT --&gt; DR_REP[reports/drift_report_top20.png]
    end

    subgraph Serving &amp;amp; API Layer
        M1 -.-&gt;|Fallback Path| API[api/main.py FastAPI]
        REG_ALIAS -.-&gt;|Primary Path| API
        API --&gt; DB[(SQLite: cases.db)]
    end

    subgraph User &amp;amp; Investigation Layer
        UI[frontend/dashboard.html] &lt;--&gt;|HTTP / REST| API
        API --&gt; SC[Scoring &amp;amp; SHAP Engine]
        API --&gt; AG[src/agent.py Investigation Copilot]
        AG --&gt; OLLAMA[Ollama: llama3.1:8b]
        AG --&gt; VAL[Hardened Automated Validator]
        VAL --&gt; CASE[Structured Case File]
        CASE --&gt; DB
        CASE --&gt; UI
    end
```

---

3. Data Ingestion & Preprocessing Architecture

3.1 Leakage-Safe Time-Based Partitioning

Random *k*-fold cross-validation allows past transactions to be predicted using future behavioral statistics, introducing lookahead leakage. FraudLens enforces chronological partitioning:

```
Timeline: ■■■
```

$\mu_{\text{amt}} = \frac{1}{|E|} \sum_{i \in E} \text{TransactionAmt}_i$

$\sigma_{\text{amt}} = \sqrt{\frac{1}{|E|} \sum_{i \in E} (\text{TransactionAmt}_i - \mu_{\text{amt}})^2}$

Where $E = \{ \text{transactions} \mid \text{card1}_i = \text{card1}_t \land \text{addr1}_i = \text{addr1}_t \land \text{DT}_i \le \text{DT}_t \}$.

---

4. Machine Learning & Explainability Pipeline

```
sequenceDiagram
    autonumber
    actor Analyst as Compliance Analyst
    participant UI as React Dashboard
    participant API as FastAPI (api/main.py)
    participant Model as LightGBM Classifier
    participant SHAP as TreeExplainer
    participant DB as SQLite (cases.db)

    Analyst-&gt;&gt;UI: Enter TransactionID &amp; Click &quot;Get Score&quot;
    UI-&gt;&gt;API: POST /score/{transaction_id}
    API-&gt;&gt;API: Query combined transaction store
    alt Transaction Not Found
        API--&gt;&gt;UI: 404 Not Found {&quot;detail&quot;: &quot;TransactionID not found&quot;}
    else Transaction Exists
        API-&gt;&gt;Model: Predict probability P(isFraud = 1 | X)
        Model--&gt;&gt;API: risk_score (e.g. 0.75231)
        API-&gt;&gt;SHAP: tree_explainer.shap_values(X)
        SHAP--&gt;&gt;API: Local feature attributions (phi_i)
        API-&gt;&gt;API: Sort abs(phi_i) descending -&gt; Top 5 features
        API-&gt;&gt;API: Compare risk_score &gt;= threshold (0.129365)
        API--&gt;&gt;UI: ScoreResponse {risk_score, flagged, threshold, top_reason_codes}
        UI-&gt;&gt;Analyst: Render ScoreCard &amp; SHAP Reason Bars
    end
```

4.1 Threshold Selection under Asymmetric Loss

Because false positives inflict analyst fatigue and legitimate customer churn, FraudLens fixes the operational threshold τ at a strict **2.0% false-positive budget** on the validation set:

$\tau = \text{Percentile}_{98} \left(\{ \hat{p}_i \mid y_i = 0, i \in \text{Validation} \} \right)$

For IEEE-CIS, this empirically yields $\tau = 0.129365$. On the held-out test set, this fixed threshold intercepts 53.39% of all fraud (Recall) at a 43.95% Precision and 2.46% empirical FP rate.

---

5. Agentic AI Investigation Architecture (`src/agent.py`)

The investigation copilot is deliberately tool-scoped and read-only. It gathers evidence deterministically, invokes a local LLM for narrative formatting, validates all claims against ground truth, and falls back gracefully upon failure.

```
flowchart TD
    Start([Investigate Triggered: TransactionID]) --&gt; Tools[Deterministic Evidence Assembly]

    subgraph Evidence Tools
        Tools --&gt; T1[get_entity_history: Point-in-Time card1+addr1 Stats]
        Tools --&gt; T2[get_related_entities: Distinct Domain Velocity]
        Tools --&gt; T3[Policy Query Builder: Evidence-Derived Query]
        T3 --&gt; T4[retrieve_policy_clause: TF-IDF Cosine Similarity]
    end

    T1 &amp;amp; T2 &amp;amp; T4 --&gt; CheckEntity{Valid Entity History Available?}

    CheckEntity -- No (Missing card1/addr1) --&gt; FastFallback[Fast Path: Code-Generated Insufficient Data File]
    FastFallback --&gt; FinalCase[Final Structured Case File]

    CheckEntity -- Yes --&gt; Prompt[Construct Single Evidence Prompt]
    Prompt --&gt; LLM[Call Local Ollama: Llama 3.1 8B]

    LLM --&gt; CleanCheck{Passes _model_text_clean?}

    subgraph Hardened Automated Validator
        CleanCheck -- Check 1 --&gt; V1[All numbers exist in get_entity_history / get_related_entities]
        CleanCheck -- Check 2 --&gt; V2[No numbers from policy text or notes]
        CleanCheck -- Check 3 --&gt; V3[No banned evaluative words: suspicious, concerning, etc.]
        CleanCheck -- Check 4 --&gt; V4[Comparative framing requires &gt;= 2 distinct grounded numbers]
    end

    CleanCheck -- Pass --&gt; Assemble[Assemble Full Case File]
    CleanCheck -- Fail (Attempt 1) --&gt; RetryPrompt[Append Retry Note &amp;amp; Prompt LLM Once More]
    RetryPrompt --&gt; LLM
    CleanCheck -- Fail (Attempt 2) --&gt; CodeFallback[Fallback: Generate Facts from Tool Data]

    LLM -.-&gt;|CUDA / Socket Crash| LLMFallback[LLM Unavailable: Assemble Pure Tool Case File]
    LLMFallback --&gt; FinalCase

    CodeFallback --&gt; Assemble
    Assemble --&gt; FinalCase
    FinalCase --&gt; SaveDB[(Persist to SQLite: cases.db)]
```

5.1 The Four Guardrails Against Hallucination

1. **Point-in-Time Temporal Gate:** `get_entity_history` filters strictly to `TransactionDT <= this transaction`. Transactions occurring at the same timestamp or later are excluded, preventing current and future label leakage.
2. **Tuple-Scoped Granularity:** `get_related_entities` groups by `(card1, addr1)` together rather than `addr1` alone, preventing regional code collisions.
3. **Evidence-Driven Policy Retrieval:** Policy search queries are constructed from actual tool counts (e.g. `distinct_email_domains_last_30_days > 3`), and a relevance floor (0.15 cosine similarity) prevents weak matches from being cited.
4. **Automated Validator (`_model_text_clean`):** Rejects comparative framing (*greater than, less than, higher than, lower than, more than, above, below, compared to, versus, relative to, than the overall/average/typical/mean*) unless the same sentence contains at least two distinct grounded numbers.

---

6. Serving Backend Architecture (`api/main.py`)

The FastAPI service exposes 5 asynchronous endpoints, utilizing a startup lifecycle (`lifespan`) to load models and data into a shared memory state dictionary (`_state`).

6.1 Dual Model-Loading Strategy

```
Application Startup:
       ■
       ▼
Try: MLflow Registry Lookup
&amp;quot;models:/fraudlens-ieee-lightgbm@production&amp;quot;
       ■
       ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
       ▼ (Success)                    ▼ (Exception / Missing URI)
Loaded from Registry            Log Warning &amp;amp; Load Joblib:
_state[&amp;quot;model_source&amp;quot;] =        &amp;quot;models/ieee_lightgbm_tuned.joblib&amp;quot;
&amp;quot;registry:...@production&amp;quot;       _state[&amp;quot;model_source&amp;quot;] = &amp;quot;joblib-fallback&amp;quot;
```

6.2 Endpoint Inventory

Endpoint	HTTP Method	Request Input	Response Output	Description
`/health`	`GET`	None	`HealthResponse`	Reports service status, model load flag, model type, and `model_source`.
`/score/{id}`	`POST`	`transaction_id: int`	`ScoreResponse`	Returns risk probability, boolean `flagged`, threshold used, and top-5 SHAP reason codes.
`/investigate/{id}`	`POST`	`transaction_id: int`	`InvestigateResponse`	Executes scoring and agent investigation, logs record to SQLite, returns full case file and `case_id`.
`/cases`	`GET`	`limit: int = 20`	`list[CaseSummary]`	Lists recent investigations from SQLite ordered by descending ID.
`/cases/{id}`	`GET`	`case_id: int`	`CaseDetail`	Fetches complete historical case record including reason codes and case file.

---

7. Database Entity-Relationship Specification

FraudLens uses a zero-configuration SQLite database (`cases.db`) configured with `PRAGMA journal_mode=WAL` for robust concurrent reads.

```
erDiagram
    CASES {
        INTEGER id PK &amp;quot;Auto-incrementing primary key&amp;quot;
        INTEGER transaction_id &amp;quot;ID of the evaluated transaction&amp;quot;
        REAL risk_score &amp;quot;Predicted fraud probability [0.0, 1.0]&amp;quot;
        REAL threshold_used &amp;quot;Operational decision threshold used&amp;quot;
        INTEGER flagged &amp;quot;1 if risk_score &amp;gt;= threshold, else 0&amp;quot;
        TEXT top_reason_codes &amp;quot;JSON array of top-5 SHAP features&amp;quot;
        TEXT case_file &amp;quot;Full multi-section text case file&amp;quot;
        TEXT created_at &amp;quot;ISO-8601 UTC timestamp&amp;quot;
    }
```

---

8. MLOps & Drift Architecture (`src/drift_check.py`)

The Population Stability Index (PSI) evaluates distributional stability across all 545 modeling features between the baseline training distribution (*P*) and the temporal validation split (*Q*):

$\text{PSI} = \sum_{b=1}^{B} \left(Q_b - P_b \right) \times \ln\left(\frac{Q_b}{P_b} \right)$

```
Quantile Binning on Reference (Train):
  Bin 1      Bin 2      Bin 3            Bin B
[■■■■■■■■■■][■■■■■■■■■■][■■■■■■■■■■■] ... [■■■■■■■■■■■]
    ■          ■          ■                 ■
Compare observed proportion in Validation:
   Q_1        Q_2        Q_3               Q_B
    ■          ■          ■                 ■
    ■■■■■■■■■■■
```

PART III: MACHINE LEARNING & EXPLAINABILITY DEEP DIVE

FraudLens — Machine Learning & Explainability Deep Dive

Author: Dhanraj Shitole

Target Audience: Machine Learning Engineers, Data Scientists, Academic Evaluators

Scope: Algorithms, Data Engineering, Hyperparameter Optimization, TreeSHAP, and Empirical Evaluation

---

1. The Machine Learning Problem Formulation

Financial fraud detection is formulated as a **cost-sensitive binary classification problem** under extreme class imbalance. Let:

- $\mathcal{X} \subset \mathbb{R}^d$ denote the d-dimensional feature space ($d = 545$).
- $\mathcal{Y} \in \{0, 1\}$ denote the binary ground-truth label, where $y = 1$ indicates confirmed fraudulent activity and $y = 0$ indicates legitimate commercial activity.
- The machine learning model parameterized by θ produces an estimated fraud probability:

$$\hat{p}(x) = P(Y = 1 \mid X = x; \theta) \in [0, 1]$$

- An operational decision rule maps the probability to a discrete action via threshold τ:

$$\hat{y}(x) = \begin{cases} 1 & \text{if } \hat{p}(x) \ge \tau \quad (\text{Flag for manual compliance review}) \\ 0 & \text{if } \hat{p}(x) < \tau \quad (\text{Auto-approve / Process transaction}) \end{cases}$$

The Asymmetric Cost Matrix

In traditional symmetric machine learning (e.g. image classification), False Positives (FP) and False Negatives (FN) incur identical unit penalties. In financial transactions:

- **False Negative Cost (C_{FN}):** Direct financial chargeback, interchange penalties, loss of goods, and compliance sanctions. $C_{\text{FN}} \propto \text{TransactionAmt}$.
- **False Positive Cost (C_{FP}):** Human analyst review cost ($\approx$ \10 - \30 per manual case), customer friction, and merchant drop-off.

Because $C_{\text{FN}} \gg C_{\text{FP}}$, optimizing standard accuracy is mathematically invalid. A trivial dummy model that predicts $\hat{y} = 0$ for all transactions achieves **96.5% accuracy** on IEEE-CIS and **99.87% accuracy** on PaySim, while intercepting **0% of fraud**.

---

2. Benchmark Datasets & Temporal Partitioning

Parameter	IEEE-CIS Fraud Detection	PaySim Mobile Money
Domain	E-commerce credit card transactions	Synthetic mobile money transfers
Total Transactions	590,540	6,362,620
Fraud Cases	20,663	8,213
Class Imbalance	27.4 : 1 (3.50% fraud)	1,221.6 : 1 (0.13% fraud)
Split Strategy	70% Train / 15% Val / 15% Held-Out Test	70% Train / 15% Val / 15% Held-Out Test
Train Observations	413,378 rows	4,453,834 rows
Validation Observations	88,581 rows	954,393 rows
Held-Out Test Observations	88,581 rows	954,393 rows
Partitioning Variable	Chronological on `TransactionDT`	Chronological on temporal `step`

Why Chronological Splitting Is Mandatory

If random *k*-fold cross-validation is applied to temporal financial transactions:

1. An entity that commits multiple fraudulent transactions over a 48-hour window would have transactions scattered across both training and validation sets.
2. The tree model would memorize the entity identifier (`card1`, `addr1`) and predict validation fraud based on future observations of the same card.
3. This creates massive **lookahead leakage**, resulting in an artificially inflated validation score that collapses in production.

In FraudLens, all encoders, imputers, and SMOTE resamplers are **fit strictly on the 70% training split**.

---

3. Feature Engineering Architecture (`src/features.py`)

Starting from raw tabular schemas, FraudLens engineers 576 modeling features (pruned to 545 non-redundant candidates):

3.1 Temporal Feature Transformations

`TransactionDT` represents elapsed seconds from a base reference date:

- Day-of-week: $D = \lfloor \frac{\text{TransactionDT}}{86400} \rfloor \pmod 7$
- Hour-of-day: $H = \lfloor \frac{\text{TransactionDT}}{3600} \rfloor \pmod{24}$

3.2 High-Cardinality Frequency Encoding

Categorical features (`card1`, `card2`, `addr1`, `P_emaildomain`, `R_emaildomain`) contain tens of thousands of unique categories. One-hot encoding would explode dimensionality and sparsify tree splits. We compute frequency encodings strictly on the training partition:

$$\text{freq}(c) = \frac{\sum_{i \in \text{Train}} \mathbb{I}(X_i = c)}{|\text{Train}|}$$

Unseen categories appearing in validation or test are assigned the fallback minimum frequency.

3.3 Composite Entity Aggregations

Device IDs and IP addresses suffer from up to 80% missingness in production payment gateways. FraudLens constructs composite behavioral entities using the tuple:

$$\text{Entity} = (\text{card1}, \text{addr1})$$

Where `card1` identifies the payment card issuer bank and cardholder series, and `addr1` represents the geographic billing district code. We compute cumulative point-in-time statistics:

1. `card1_addr1_count`: Number of historical transactions on this exact card and address.
2. `card1_addr1_amt_mean`: Running mean transaction amount for this entity.

3. `card1_addr1_amt_std`: Spending standard deviation (captures anomalous deviation from typical spend).
4. `card1_addr1_email_count`: Number of distinct email domains ever associated with this card+address.

---

4. The Anomaly Detection Fallacy (PaySim Benchmark)

A common assumption in academic literature asserts that when fraud imbalance exceeds 1,000:1, unsupervised anomaly detection (e.g. Isolation Forest) should outperform supervised classifiers because positive labels are too scarce to learn decision boundaries.

FraudLens tested this hypothesis directly on PaySim's 6.36 million transactions (1,221.6:1 imbalance):

```
PaySim Evaluation: Supervised LightGBM vs. Unsupervised Isolation Forest
■■■
```

```
Feature 2
   ▲
   ■        o (Synthetic Fraud)
   ■     x ■■■■■■■■ o ■■■■■■■■ x (Minority Fraud Instances)
   ■        \
   ■         x
   ■
   ■   •   •   •   •   • (Majority Non-Fraud Instances)
   ■■■■■■■■
```

$\text{top_reasons} = \left\{ (\text{feature}_j, \text{value}_j, \phi_j) \mid j \in \text{rank}[:5] \right\}$

---

7. Comprehensive Model Benchmark & Held-Out Test Evaluation

7.1 Validation Set Benchmark (IEEE-CIS, *N=88,581*)

Model Architecture	PR-AUC	ROC-AUC	Precision @ 2% FP	Recall @ 2% FP	F1 @ Budget
Logistic Regression (Class-Weighted)	0.3050	0.8180	0.3740	0.3360	0.3540
LightGBM + scale_pos_weight	0.5520	0.9160	0.4850	0.5290	0.5060
LightGBM + SMOTE (Phase 4 Baseline)	0.5566	0.9042	0.4900	0.5410	0.5150
LightGBM + SMOTE Tuned (Phase 5, Serving)	**0.5975**	**0.9242**	**0.5071**	**0.5786**	**0.5405**

Key Result: Hyperparameter tuning yielded a **+7.3% relative lift** in PR-AUC (from 0.5566 to 0.5975) and increased operational recall at the 2% FP budget from 54.1% to 57.86%.

7.2 Final Held-Out Test Evaluation (IEEE-CIS, *N=88,581*)

Evaluated strictly one-shot on unseen future data with the decision threshold frozen at $\tau = 0.129365$:

Evaluation Metric	Validation Set	Held-Out Test Set	Delta (Δ)	Analysis
Rows (*N*)	88,581	88,581	0	Equal sample sizes
Fraud Rate (Base Rate)	3.4341%	3.4804%	+0.0463 pp	Stable base prevalence
PR-AUC	**0.5975**	**0.5324**	-0.0651	Expected mild degradation on unseen future data
PR-AUC Lift over Base Rate	17.40x	15.30x	-2.10x	Substantial discriminative power retained
ROC-AUC	0.9242	0.9066	-0.0176	Excellent global ranking preserved
Precision @ Threshold	50.71%	43.95%	-6.76 pp	44% of flagged alerts are true fraud
Recall @ Threshold	57.86%	53.39%	-4.47 pp	Over 53% of all fraud intercepted
F1 Score @ Threshold	0.5405	0.4821	-0.0584	Strong operational performance
Actual False-Positive Rate	**2.0000%**	**2.4550%**	+0.455 pp	Slightly exceeded 2.0% budget due to mild drift
% of All Transactions Flagged	3.92%	4.23%	+0.31 pp	Manageable queue volume for human analysts

7.3 Calibration & Brier Skill Score (`reports/calibration_test.png`)

A model predicting $P(\text{fraud}) = 0.80$ should ideally see 80 out of 100 such transactions be actual fraud.

- **Mean Predicted Probability:** 3.19%
- **Observed Fraud Prevalence:** 3.48%
- **Brier Score:** $BS = \frac{1}{N} \sum_{i=1}^{N} (\hat{p}_i - y_i)^2 = \mathbf{0.02213}$
- **Baseline Brier Score (Constant Base Rate):** $BS_{\text{base}} = \bar{y}(1 - \bar{y}) = \mathbf{0.03359}$
- **Brier Skill Score:**

$\text{BSS} = 1 - \frac{BS}{BS_{\text{base}}} = 1 - \frac{0.02213}{0.03359} = \mathbf{0.34115} \quad (\mathbf{34.12\%})$

A Brier Skill Score of 34.12% verifies strong probabilistic accuracy and calibration over uninformative baselines.

PART IV: AI AGENT & LLM COPILOT DEEP DIVE

FraudLens — Agentic AI & Investigation Copilot Deep Dive

Author: Dhanraj Shitole

Target Audience: AI Engineers, LLM System Architects, Compliance Officers

Source Code: `src/agent.py`

Evaluation Logs: `reports/`

---

1. Introduction: Generative AI in High-Stakes Compliance

In banking and fraud prevention, deploying unconstrained Large Language Models (LLMs) as autonomous decision-makers is fundamentally unacceptable due to legal, regulatory, and financial compliance requirements:

1. **Hallucination Risk:** Generative language models predict token sequences based on distributional probabilities, not logical truth. In compliance reports, fabricating an entity transaction count or a policy violation can expose an institution to regulatory fines.
2. **Temporal Label Leakage:** In naive tool calling, an LLM querying a customer's history might inadvertently observe future confirmed fraud flags that occurred days *after* the transaction being evaluated, silently leaking the target label.
3. **Sycophancy & Drift:** An LLM prompted with an elevated fraud score tends to rationalize that score, creating fabricated justifications to agree with the model rather than assessing raw facts objectively.

FraudLens resolves this through an evidence-grounded agent architecture governed by one non-negotiable rule:

> ***Code makes decisions, models score, and language models write prose.***

> *The LLM is never permitted to decide whether a transaction is fraudulent, never selects the policy action, and is strictly restricted to drafting two narrative sentences whose numerical contents are validated against tool output.*

---

2. Agent Redesign: From Multi-Turn Tool Calling to Single-Turn Synthesis

2.1 The Latency & Reliability Problem in Multi-Turn Tool Calling

In early prototypes (Evaluation Round 1), FraudLens followed standard multi-turn tool calling:

- Round 1: Model receives Transaction ID and decides whether to call `get_entity_history` or `get_related_entities`.
- Round 2: Model receives tool outputs and writes a search query to call `retrieve_policy_clause`.
- Round 3: Model receives policy text and outputs a 5-section case file.

Evaluation revealed fatal flaws in this multi-turn pattern:

1. **Tool Invocation Failure:** Small local models (Llama 3.1 8B) frequently emitted raw JSON strings in plain text instead of executing native tool calling, causing parsing crashes.
2. **Blind Policy Retrieval:** In Round 2, the model attempted to write a policy search query *before* fully evaluating entity evidence. It searched generically (e.g., `"foreign transaction fraud"`), TF-IDF matched an irrelevant clause (e.g., cross-border policy), and the model rationalized the mismatch.
3. **Severe Latency:** Executing 3 sequential model completions on consumer hardware (especially CPU-mode Ollama) required 45 to 90 seconds per investigation.

2.2 The Single-Turn Deterministic Pipeline

Empirical validation showed that human analysts required all three evidence categories in 100% of cases, and the model's actual contribution was confined to two narrative sections:

- `ENTITY SUMMARY`
- `TOP RISK FACTORS`

All remaining sections (`RELATED ENTITY SIGNAL`, `RELEVANT POLICY`, `RECOMMENDED ACTION`) are purely deterministic logic.

The Redesigned Architecture:

1. Deterministic Python executes `get_entity_history` and `get_related_entities` up front.
2. Deterministic Python builds the policy search query directly from tool counts and queries TF-IDF with a relevance floor.
3. If the entity lacks identifiers (`card1` or `addr1` is NaN), the system immediately outputs an "insufficient data" template in **0 model calls**.
4. If entity data exists, a single structured prompt is sent to Llama 3.1 8B asking for exactly two lines.
5. An automated validator inspects the response; if clean, the case file is assembled. If invalid, the model receives one retry; if still invalid, the system falls back to tool-derived bullets.

Latency Result: Reduced model calls from 2–3 down to **exactly 1**, slashing runtime by over 65% with zero degradation in analytical quality.

---

3. The Four Hallucination Classes & How They Were Solved

During iterative validation against held-out fraud cases, four distinct hallucination classes were discovered and systematically eliminated:

Hallucination Class 1: Geographic Hallucination

- **Failure Mode:** The model read the `addr1` column (e.g. `addr1 = 299`) and wrote: *"The transaction originated from country 299, indicating an unauthorized cross-border payment."*
- **Root Cause:** `addr1` in IEEE-CIS is an anonymized internal billing zip/region code, not an ISO country code.
- **Fix:** Explicit system prompt constraint prohibiting geographic inferences: *"addr1 is a coarse internal billing region code — NOT a country, street address, or geographic identifier. Never call it a country and never claim cross-border activity."*

Hallucination Class 2: Data-Granularity Aggregation Bug

- **Failure Mode:** The `get_related_entities` tool originally grouped transactions on `addr1` alone, reporting: *"723 other cards share this address."*
- **Root Cause:** In IEEE-CIS, thousands of distinct cardholders share coarse billing region `299`. Grouping by `addr1` alone produced meaningless false fraud-ring signals.
- **Fix:** Redesigned tool to group strictly by the composite tuple `(card1, addr1)` together, matching the granularity of engineered modeling features.

Hallucination Class 3: Temporal Label Leakage & Window Overreach

- **Failure Mode:** The model asserted: *"Entity had 5 transactions within a 24-hour window, violating Policy POL-001."* However, the tool only reported lifetime totals.
- **Root Cause:** The entity history tool lacked point-in-time window filtering. Furthermore, calculating prior fraud originally included the transaction being investigated, silently leaking the label.
- **Fix:**

1. Gated all tool queries strictly to `TransactionDT <= this_transaction.TransactionDT`.
2. Prior fraud count strictly requires `TransactionDT < this_transaction.TransactionDT` and `TransactionID != this_transaction.TransactionID`.
3. Added explicit 24-hour and 30-day window calculators using `SECONDS_24H` (*86,400*) and `SECONDS_30D` (*2,592,000*).

Hallucination Class 4: Vacuous Comparative Framing

- **Failure Mode:** The model wrote: *"The mean transaction amount of 144.14 is greater than the overall mean of 144.14."*

- **Root Cause:** The number was technically present in the evidence tool output, and the phrasing used no banned words, so early validators let it through. However, the comparison was logically vacuous and self-referential.
- **Fix:** Hardened validator with regex `_COMPARATIVE_RE`. Any sentence containing comparative phrasing (*greater than, less than, higher than, lower than, more than, above, below, compared to, versus, relative to, than the overall/average/typical/mean*) is rejected unless the sentence contains at least **two distinct grounded numbers**.

---

4. The Three Read-Only Deterministic Tools

4.1 `get_entity_history(transaction_id: int) -> dict`

Extracts point-in-time card+address transaction metrics as of the investigated transaction:


```
{
    &quot;transaction_id&quot;: 3141064,
    &quot;entity_key&quot;: &quot;card1=100, addr1=200&quot;,
    &quot;total_transactions_by_entity&quot;: 12,
    &quot;total_amount_by_entity&quot;: 1729.68,
    &quot;mean_amount_by_entity&quot;: 144.14,
    &quot;known_fraud_count_for_entity&quot;: 1,
    &quot;this_transaction_amount&quot;: 350.00,
    &quot;transactions_last_24_hours&quot;: 6,
    &quot;note&quot;: &quot;Totals cover this entity up to and including this transaction only...&quot;
}
```


4.2 `get_related_entities(transaction_id: int) -> dict`

Evaluates multi-identity clustering across the exact same card and billing address tuple:


```
{
    &quot;transaction_id&quot;: 3141064,
    &quot;entity_key&quot;: &quot;card1=100, addr1=200&quot;,
    &quot;distinct_email_domains_for_this_card_and_address&quot;: 4,
    &quot;distinct_email_domains_last_30_days&quot;: 4,
    &quot;total_transactions_for_this_exact_entity&quot;: 12,
    &quot;note&quot;: &quot;Counts cover transactions up to and including this one...&quot;
}
```


4.3 `retrieve_policy_clause(query: str) -> dict`

Applies TF-IDF vectorization and cosine similarity over the six enterprise fraud policy clauses:

- `POL-001`: Shared Card-Address Velocity (>5 txns in 24h).
- `POL-002`: New Account High-Value Transaction (>$500 within 7 days).
- `POL-003`: Email Domain Fraud Ring Indicator (>3 distinct email domains in 30 days).
- `POL-004`: Product Category Risk Weighting (Digital goods lower tolerance).
- `POL-005`: Repeat False-Positive Entities (>2 cleared false-positives in 90 days).
- `POL-006`: Cross-Border Transaction Handling (Billing country mismatch).

Relevance Floor: If maximum cosine similarity < 0.15, the system returns `policy_id: null`, preventing spurious policy citations.

---

5. Hardened Automated Validator Implementation

Before any text authored by Llama 3.1 8B is incorporated into a case file, it must pass `_model_text_clean(text, tool_results)`:

```
def _model_text_clean(text: str, tool_results: dict) -&gt; bool:
    if not text:
        return False
    # 1. Banned evaluative / speculative words
    if _BANNED_TEXT_RE.search(text) or _SNAKE_RE.search(text):
        return False
    # 2. Number grounding in evidence tools
    grounded = _grounded_numbers(tool_results)
    if not _numbers_in(text) &lt;= grounded:
        return False
    # 3. Temporal window checks
    n24 = tool_results.get(&quot;get_entity_history&quot;, {}).get(&quot;transactions_last_24_hours&quot;)
    d30 = tool_results.get(&quot;get_related_entities&quot;, {}).get(&quot;distinct_email_domains_last_30_days&quot;)
    for chunk in re.split(r&quot;(?&lt;=[.!?])\s+|\n&quot;, text):
        nums = _numbers_in(chunk)
        if _WINDOW_30D_RE.search(chunk) and not (isinstance(d30, int) and float(d30) in nums):
            return False
        if _WINDOW_24H_RE.search(chunk) and not (isinstance(n24, int) and float(n24) in nums):
            return False
        # 4. Comparative framing guard: requires &gt;= 2 distinct grounded numbers
        if _COMPARATIVE_RE.search(chunk):
            sentence_grounded = nums &amp; grounded
            if len(sentence_grounded) &lt; 2:
                return False
    return True
```

Numerical Grounding Isolation (`_grounded_numbers`)

To prevent the model from citing numbers found in policy text or system notes:

- The entire **`retrieve_policy_clause`** subdictionary is skipped.
- Any dictionary key named **`"note"`** at any nesting level is skipped.
- Numbers embedded inside the **`entity_key`** string (e.g. **`card1=100, addr1=200`**) are accepted as grounded identifiers.

---

6. Deterministic Case File Synthesis & Resilience

6.1 The 5 Required Sections

The final case file returned to the analyst strictly adheres to standard compliance reporting:

1. **`ENTITY SUMMARY:`** Narrative synthesis of card+address transaction volume and spend.
2. **`TOP RISK FACTORS:`** Bulleted literal facts from evidence.
3. **`RELATED ENTITY SIGNAL:`** Written deterministically by code from **`get_related_entities`**.
4. **`RELEVANT POLICY:`** Written deterministically by code, displaying clause ID, title, matched threshold, and verification status.
5. **`RECOMMENDED ACTION:`** Written deterministically by code:

- If prior confirmed fraud *> 0* OR any policy is triggered *\to* **`"Escalate for manual review — [reasons]."`**
- Otherwise *\to* **`"Monitor, no immediate action — no policy clause is triggered and this entity has 0 prior confirmed fraud cases."`**

6.2 Zero-Data-Loss Failure Fallback

If Ollama is stopped, or if the model fails validation on both attempt 1 and retry attempt 2:

The agent catches **`_LLMError`** and immediately synthesizes a fallback case file using **`_fallback_case_file(tool_results)`**:

- Prepends: **`[NOTE: automated LLM synthesis failed validation / language model was unavailable; this case file was assembled directly from tool outputs with no model-written text.]`**
- Automatically fills **`ENTITY SUMMARY`** and **`TOP RISK FACTORS`** using raw Python templates populated with grounded tool data.
- The analyst receives a complete, formatted case file with 100% accurate evidence.

PART V: REST API & INTEGRATION REFERENCE

FraudLens — REST API Reference & Specification

Service Name: FraudLens Scoring & Investigation API

Default Base URL: `http://127.0.0.1:8000`

Framework: FastAPI 0.136.1 (ASGI)

OpenAPI Specification: Available interactively at `/docs` (Swagger UI) and `/redoc` (ReDoc)

---

1. Overview & General Standards

The FraudLens backend exposes RESTful endpoints supporting real-time fraud probability scoring, game-theoretic SHAP reason-code decomposition, and agentic investigation synthesis.

Core Protocol Principles:

- **Transport:** HTTP / 1.1 JSON over TCP.
- **Content-Type:** `application/json` for all requests and responses.
- **Cross-Origin Resource Sharing (CORS):** Enabled for all origins (`allow_origins=["*"]`) to allow the local browser-based analyst console (`frontend/dashboard.html`) to access endpoints without CORS restrictions.
- **Data Lookup Scope:** In the current demonstration version, transactions are queried by `TransactionID` from the combined IEEE-CIS processed dataset (train + validation, totaling 501,959 transactions).

---

2. Global Error Handling & HTTP Status Codes

The API returns standard RFC 7807 problem details in error conditions:

Status Code	Meaning	Typical Trigger
200 OK	Successful execution	Transaction scored, case retrieved, or health check passed.
404 Not Found	Resource missing	`TransactionID` does not exist in dataset, or `case_id` does not exist in SQLite.
500 Internal Error	Module failure	Missing agent module or database connection failure.
503 Service Unavailable	Dependency unavailable	Local Ollama service is stopped, unresponsive, or crashed during LLM generation.

Error Response Format:

```
{
  &amp;quot;detail&amp;quot;: &amp;quot;TransactionID 9999999999 not found in dataset.&amp;quot;
}
```

---

3. Endpoint Specifications

3.1 Service Health Check

`GET /health`

Verifies that the API service is operational and reports which model backend is active.

- **Request Headers:** None required.
- **Request Parameters:** None.
- **Response Model:** `dict`

Success Response (`200 OK`):

```
{
  &amp;quot;status&amp;quot;: &amp;quot;ok&amp;quot;,
  &amp;quot;model_loaded&amp;quot;: true,
  &amp;quot;model_type&amp;quot;: &amp;quot;LightGBM (tuned, Phase 5 hyperparameters, SMOTE-trained)&amp;quot;,
  &amp;quot;model_source&amp;quot;: &amp;quot;registry:fraudlens-ieee-lightgbm@production&amp;quot;
}
```

Note on `model_source`*:* If MLflow registry resolution succeeds, `model_source` will indicate the registry model name and alias (`registry:fraudlens-ieee-lightgbm@production`). If the registry path cannot be resolved (such as within a Docker container pointing to a local host path), the service falls back gracefully to `joblib-fallback`.

Example cURL:

```
curl -X GET http://127.0.0.1:8000/health
```

3.2 Real-Time Transaction Scoring

`POST /score/{transaction_id}`

Computes the fraud probability for the specified `TransactionID` using the tuned LightGBM model, applies the 2% false-positive budget threshold, and calculates the top-5 feature contributions using TreeSHAP.

- **Path Parameter:**
- `transaction_id` (`integer`, required): The unique integer ID of the transaction (e.g. `3141064`).
- **Request Body:** None.
- **Response Model:** `ScoreResponse`

Response Schema (`ScoreResponse`):


```
{
  &amp;quot;transaction_id&amp;quot;: 3141064,
  &amp;quot;risk_score&amp;quot;: 0.81245,
  &amp;quot;flagged&amp;quot;: true,
  &amp;quot;threshold_used&amp;quot;: 0.12937,
  &amp;quot;top_reason_codes&amp;quot;: [
    {
      &amp;quot;feature&amp;quot;: &amp;quot;card1_addr1_amt_mean&amp;quot;,
      &amp;quot;value&amp;quot;: 144.14,
      &amp;quot;shap_contribution&amp;quot;: 0.32145
    },
    {
      &amp;quot;feature&amp;quot;: &amp;quot;card1_addr1_count&amp;quot;,
      &amp;quot;value&amp;quot;: 12.0,
      &amp;quot;shap_contribution&amp;quot;: 0.19842
    },
    {
      &amp;quot;feature&amp;quot;: &amp;quot;C13&amp;quot;,
      &amp;quot;value&amp;quot;: 5.0,
      &amp;quot;shap_contribution&amp;quot;: 0.14231
    },
    {
      &amp;quot;feature&amp;quot;: &amp;quot;TransactionAmt&amp;quot;,
      &amp;quot;value&amp;quot;: 350.0,
      &amp;quot;shap_contribution&amp;quot;: 0.08912
    },
    {
      &amp;quot;feature&amp;quot;: &amp;quot;dist1&amp;quot;,
      &amp;quot;value&amp;quot;: 18.0,
      &amp;quot;shap_contribution&amp;quot;: -0.04123
    }
  ]
}
```


Field Explanations:

- **`risk_score`**: Calibrated probability $P(\text{isFraud} = 1 \mid X)$ rounded to 5 decimal places.
- **`flagged`**: Boolean indicator; **`true`** if **`risk_score >= threshold_used`**, otherwise **`false`**.
- **`threshold_used`**: The operational decision threshold established on the validation split at a 2.0% false-positive budget ($\tau = 0.129365$).
- **`top_reason_codes`**: Array of the 5 features with the highest absolute Shapley values ($|\phi_i|$).
- **`feature`**: Exact name of the modeling feature.
- **`value`**: Raw or imputed numerical value for this transaction.
- **`shap_contribution`**: Additive margin contribution ϕ_i. Positive values push risk higher; negative values push risk lower.

Error Responses:

- **`404 Not Found`**:


```
  {
    &amp;quot;detail&amp;quot;: &amp;quot;TransactionID 9999999999 not found in dataset.&amp;quot;
  }
```


Example cURL:

```
curl -X POST http://127.0.0.1:8000/score/3141064
```

3.3 Trigger Agent Investigation

`POST /investigate/{transaction_id}`

Executes transaction scoring and triggers the AI investigation copilot (`src/agent.py`). Gathers point-in-time entity history, retrieves policy citations via TF-IDF, generates and validates narrative facts via local Llama 3.1 8B, logs the complete case file to SQLite (`cases.db`), and returns the investigation record.

- **Path Parameter:**
- `transaction_id` (`integer`, required): The unique integer ID of the transaction.
- **Request Body:** None.
- **Response Model:** `InvestigateResponse`

Response Schema (`InvestigateResponse`):


```
{
  &amp;quot;transaction_id&amp;quot;: 3141064,
  &amp;quot;risk_score&amp;quot;: 0.81245,
  &amp;quot;flagged&amp;quot;: true,
  &amp;quot;threshold_used&amp;quot;: 0.12937,
  &amp;quot;top_reason_codes&amp;quot;: [
    {
      &amp;quot;feature&amp;quot;: &amp;quot;card1_addr1_amt_mean&amp;quot;,
      &amp;quot;value&amp;quot;: 144.14,
      &amp;quot;shap_contribution&amp;quot;: 0.32145
    }
  ],
  &amp;quot;case_file&amp;quot;: &amp;quot;ENTITY SUMMARY: This card+address entity has 12 transactions totaling <font face="Helvetica-Oblique" co
  &amp;quot;case_id&amp;quot;: 1
}
```


Error Responses:

- `404 Not Found`: If the transaction is not in the dataset.
- `503 Service Unavailable`: If Ollama cannot be reached:


```
  {
    &amp;quot;detail&amp;quot;: &amp;quot;Agent investigation failed — the local Ollama service may have crashed or isn&amp;#x27;t running. Try &amp;#x27;
  }
```


Example cURL:

```
curl -X POST http://127.0.0.1:8000/investigate/3141064
```

3.4 List Investigation Cases

`GET /cases`

Retrieves a paginated list of previously logged investigation cases from SQLite (`cases.db`), ordered by descending case ID (most recent first).

- **Query Parameters:**
- `limit` (`integer`, optional, default = 20): Maximum number of case summaries to return.
- **Response Model:** `list[CaseSummary]`

Success Response (`200 OK`):


```
[
  {
    &amp;quot;id&amp;quot;: 1,
    &amp;quot;transaction_id&amp;quot;: 3141064,
    &amp;quot;risk_score&amp;quot;: 0.81245,
    &amp;quot;created_at&amp;quot;: &amp;quot;2026-11-01T10:45:00.123456+00:00&amp;quot;
  }
]
```


Example cURL:


```
curl -X GET &amp;quot;http://127.0.0.1:8000/cases?limit=10&amp;quot;
```

3.5 Retrieve Investigation Case Detail

`GET /cases/{case_id}`

Retrieves the complete historical record of a previously completed investigation, including stored risk score, threshold, reason codes, and full case file text.

- **Path Parameter:**
- `case_id` (`integer`, required): The primary key ID of the stored case.
- **Response Model:** `CaseDetail`

Success Response (`200 OK`):


```
{
  &amp;quot;id&amp;quot;: 1,
  &amp;quot;transaction_id&amp;quot;: 3141064,
  &amp;quot;risk_score&amp;quot;: 0.81245,
  &amp;quot;threshold_used&amp;quot;: 0.12937,
  &amp;quot;flagged&amp;quot;: true,
  &amp;quot;top_reason_codes&amp;quot;: [
    {
      &amp;quot;feature&amp;quot;: &amp;quot;card1_addr1_amt_mean&amp;quot;,
      &amp;quot;value&amp;quot;: 144.14,
      &amp;quot;shap_contribution&amp;quot;: 0.32145
    }
  ],
  &amp;quot;case_file&amp;quot;: &amp;quot;ENTITY SUMMARY: This card+address entity has 12 transactions totaling $1729.68...\nRECOMMENDED ACTION:
  &amp;quot;created_at&amp;quot;: &amp;quot;2026-11-01T10:45:00.123456+00:00&amp;quot;
}
```


Error Responses:

- `404 Not Found`:


```
  {
    &amp;quot;detail&amp;quot;: &amp;quot;Case 999999 not found.&amp;quot;
  }
```


Example cURL:


```
curl -X GET http://127.0.0.1:8000/cases/1
```


---

4. Client Integration Examples

Python Client (requests)

```
import requests

BASE_URL = &amp;quot;http://127.0.0.1:8000&amp;quot;

# 1. Health check
health = requests.get(f&amp;quot;{BASE_URL}/health&amp;quot;).json()
print(&amp;quot;API Status:&amp;quot;, health[&amp;quot;status&amp;quot;], &amp;quot;| Model:&amp;quot;, health[&amp;quot;model_source&amp;quot;])

# 2. Score a transaction
txn_id = 3141064
score_resp = requests.post(f&amp;quot;{BASE_URL}/score/{txn_id}&amp;quot;)
if score_resp.status_code == 200:
    data = score_resp.json()
    print(f&amp;quot;Transaction {txn_id}: Risk Score = {data[&amp;#x27;risk_score&amp;#x27;] * 100:.1f}%&amp;quot;)
    print(f&amp;quot;Flagged for Review: {data[&amp;#x27;flagged&amp;#x27;]}&amp;quot;)
    print(&amp;quot;Top Reason:&amp;quot;, data[&amp;quot;top_reason_codes&amp;quot;][0][&amp;quot;feature&amp;quot;])

# 3. Trigger Investigation
inv_resp = requests.post(f&amp;quot;{BASE_URL}/investigate/{txn_id}&amp;quot;)
if inv_resp.status_code == 200:
    case_data = inv_resp.json()
    print(&amp;quot;Investigation Case ID:&amp;quot;, case_data[&amp;quot;case_id&amp;quot;])
    print(&amp;quot;\n--- CASE FILE ---\n&amp;quot;)
    print(case_data[&amp;quot;case_file&amp;quot;])
```

JavaScript Client (Browser Fetch)

```
const API_BASE = &amp;quot;http://127.0.0.1:8000&amp;quot;;

async function scoreAndInvestigate(txnId) {
  // Score transaction
  const scoreRes = await fetch(<font face="Courier" color="#991B1B"><b><font face="Helvetica-Oblique" color="#1E3A8A"><i>{API_BASE}/sc
  if (!scoreRes.ok) throw new Error(&amp;quot;Scoring failed&amp;quot;);
  const scoreData = await scoreRes.json();
  console.log(<font face="Courier" color="#991B1B"><b>Risk: ${(scoreData.risk_score * 100).toFixed(1)}%</b></font>);

  // Investigate if flagged
  if (scoreData.flagged) {
    const invRes = await fetch(<font face="Courier" color="#991B1B"><b><font face="Helvetica-Oblique" color="#1E3A8A"><i>{API_BASE}/in
    const invData = await invRes.json();
    console.log(&amp;quot;Generated Case File:&amp;quot;, invData.case_file);
  }
}
```

PART VI: DEVOPS, MLOPS & CONTAINERIZATION GUIDE

FraudLens — DevOps, MLOps & Production Engineering Guide

Author: Dhanraj Shitole

Target Audience: DevOps Engineers, MLOps Engineers, System Administrators

Components Covered: MLflow Model Registry, PSI Drift Detection, Docker, GitHub Actions CI/CD

---

1. MLOps Architecture & Model Registry

In mission-critical financial applications, saving a model as an untracked local binary file introduces severe operational hazards:

- Lack of traceability: No audit trail connecting a serving binary to the training dataset, commit hash, or hyperparameter configuration.
- Stale deployments: Difficulty in rolling back or staging new model candidates without downtime.

FraudLens integrates **MLflow (v3.16.0)** with an embedded SQLite backend (`mlflow.db`).

1.1 Model Registration & Modern Aliasing (`src/register_model.py`)

Earlier versions of MLflow utilized stage-based promotion (`Staging`, `Production`, `Archived`). MLflow 2.9+ deprecated stages in favor of the **Model Aliases API**:

1. The tuned model is logged under experiment `FraudLens_IEEE-CIS`.
2. Model parameters (`threshold_2pct_fp_budget`, `n_features`, `model_type`) are logged alongside the model artifact.
3. The model is registered in the central registry as `fraudlens-ieee-lightgbm`.
4. The client assigns the mutable pointer `@production` to the target version:

```
   client.set_registered_model_alias(
       name=&amp;quot;fraudlens-ieee-lightgbm&amp;quot;,
       alias=&amp;quot;production&amp;quot;,
       version=latest_version
   )
```

1.2 Dual Serving Loader in `api/main.py`

The FastAPI application implements a resilient dual-path loader during startup:

```
mlflow.set_tracking_uri(f&amp;quot;sqlite:///{os.path.join(BASE_DIR, &amp;#x27;mlflow.db&amp;#x27;)}&amp;quot;)
try:
    # 1. Primary Path: Load active production model by alias
    registry_model = mlflow.lightgbm.load_model(&amp;quot;models:/fraudlens-ieee-lightgbm@production&amp;quot;)
    if not hasattr(registry_model, &amp;quot;predict_proba&amp;quot;):
        raise ValueError(&amp;quot;Registry model has no predict_proba method.&amp;quot;)
    shap.TreeExplainer(registry_model)  # Verify SHAP compatibility
    _state[&amp;quot;model&amp;quot;] = registry_model
    _state[&amp;quot;model_source&amp;quot;] = &amp;quot;registry:fraudlens-ieee-lightgbm@production&amp;quot;
except Exception as _reg_err:
    # 2. Fallback Path: Load static joblib artifact
    print(f&amp;quot;Falling back to local joblib model ({_reg_err}).&amp;quot;)
    _state[&amp;quot;model&amp;quot;] = joblib.load(os.path.join(MODELS_DIR, &amp;quot;ieee_lightgbm_tuned.joblib&amp;quot;))
    _state[&amp;quot;model_source&amp;quot;] = &amp;quot;joblib-fallback&amp;quot;
```

Why Fallback Is Critical: Inside lightweight Docker containers or remote virtual environments, SQLite tracking databases often contain host-specific absolute paths to artifacts that cannot be resolved. The fallback guarantees 100% service uptime.

---

2. Drift Monitoring via Population Stability Index (`src/drift_check.py`)

Financial transaction patterns drift over time due to seasonal consumer habits, infrastructure updates, and emerging fraud vectors. FraudLens calculates the **Population Stability Index (PSI)** to monitor feature drift without requiring heavy external dependencies (e.g., Evidently AI).

2.1 Mathematical Formulation

Let P represent the reference distribution (Training split, earliest 70%) and Q represent the comparison distribution (Validation split, subsequent 15%). Continuous features are discretized into $B = 10$ quantile bins defined strictly by the reference distribution:

$$\text{PSI} = \sum_{b=1}^{B} \left(Q_b - P_b \right) \times \ln\left(\frac{Q_b}{P_b} \right)$$

Where:

- P_b is the proportion of reference observations falling into bin b.
- Q_b is the proportion of comparison observations falling into bin b.
- Values are clipped to a floor of $\epsilon = 10^{-4}$ to prevent division by zero or undefined logarithms.

2.2 Standard Industrial Thresholds

- $\text{PSI} < 0.10$**:** Negligible shift; distribution is stable.
- $0.10 \le \text{PSI} < 0.25$**:** Moderate shift; flag for visual inspection.
- $\text{PSI} \ge 0.25$**:** Significant distribution shift; investigate root cause and trigger model retraining.

2.3 Empirical Results (Train vs. Validation Drift)

Evaluating all 545 candidate features across 501,959 transactions:

- **Stable ($\text{PSI} < 0.10$): 535 features** (98.2% of feature space).
- **Moderate Shift ($0.10 \le \text{PSI} < 0.25$): 2 features** (`id_31`, `card4`).
- **Initial Flagged Features ($\text{PSI} \ge 0.25$): 8 features**.

Diagnosing the `TransactionDT` Tautological Artifact:

The highest calculated PSI occurred on `TransactionDT` ($\text{PSI} = 8.28$). However, `TransactionDT` is the chronological time variable itself, which was explicitly used to split train (earliest 70%) from validation (next 15%). The two distributions have completely disjoint supports by mathematical definition. Reporting this as "data drift" would be a severe analytical error; our pipeline correctly isolates it as an expected temporal partitioning artifact.

Genuine Drift Candidates:

Excluding `TransactionDT`, exactly **7 features (1.3%)** exhibit genuine drift:

1. `id_13`, `id_20`, `id_02`: Device and network fingerprint headers. Plausibly explained by changes in browser versions, mobile OS updates, or telemetry SDK updates across collection epochs.
2. `V151`, `V160`, `V144`, `V143`, `V166`, `V164`: Anonymized V-series features reflecting payment gateway metadata.

Outputs: Persisted to [`reports/drift_report_train_vs_val.csv`](file:///c:/Users/DHANRAJ/fraudlens/reports/drift_report_train_vs_val.csv) and [`reports/drift_report_top20.png`](file:///c:/Users/DHANRAJ/fraudlens/reports/drift_report_top20.png).

---

3. Containerization Architecture (Docker & Docker Compose)

FraudLens provides complete local containerization for the scoring backend and database layers.

3.1 Dockerfile Specification (`Dockerfile`)

```
FROM python:3.13-slim

WORKDIR /app

# Install OS build dependencies required for compiling wheels
RUN apt-get update &amp;&amp; apt-get install -y --no-install-recommends \
        gcc \
        g++ \
        libgomp1 \
    &amp;&amp; rm -rf /var/lib/apt/lists/*

# Layer caching: Copy requirements first
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application source code only
COPY api/ api/
COPY src/ src/

EXPOSE 8000
CMD [&quot;uvicorn&quot;, &quot;api.main:app&quot;, &quot;--host&quot;, &quot;0.0.0.0&quot;, &quot;--port&quot;, &quot;8000&quot;]
```

3.2 Docker Compose (`docker-compose.yml`)

```
services:
  api:
    build: .
    ports:
      - &quot;8000:8000&quot;
    volumes:
      # Mount runtime data and models; do not bake multi-GB files into image
      - ./data:/app/data
      - ./models:/app/models
      - ./mlflow.db:/app/mlflow.db
      - ./mlruns:/app/mlruns
    environment:
      # Point to local host Ollama service
      - OLLAMA_HOST=http://host.docker.internal:11434
```

3.3 Context Optimization (`.dockerignore`)

To keep build context transfer times sub-second, multi-gigabyte datasets, models, databases, and dev caches are excluded:

```
data/
models/
mlruns/
mlflow.db
cases.db
.git
__pycache__
tests/
notebooks/
reports/
frontend/
*.pyc
.pytest_cache/
```

---

4. Continuous Integration Pipeline (GitHub Actions)

Located at [`.github/workflows/ci.yml`](file:///c:/Users/DHANRAJ/fraudlens/.github/workflows/ci.yml). Automatically triggered on every push and pull request to branch `main`.

```
flowchart TD
    Trigger([Push / PR to main]) --&gt; Job1[Job 1: Lint, Test &amp; Validate]

    subgraph Job 1 Steps
        Job1 --&gt; S1[Checkout Code via actions/checkout@v4]
        S1 --&gt; S2[Set up Python 3.13 with pip cache]
        S2 --&gt; S3[pip install -r requirements.txt pytest]
        S3 --&gt; S4[python -m compileall src/ api/ tests/]
        S4 --&gt; S5[Validate frontend/dashboard.html syntax]
        S5 --&gt; S6[pytest tests/ -v - 51 Tests]
    end

    Job1 --&gt;|Success| Job2[Job 2: Container Build Verification]

    subgraph Job 2 Steps
        Job2 --&gt; B1[Setup Docker Buildx]
        B1 --&gt; B2[docker build -t fraudlens-api:latest .]
    end
```

---

5. Production Cloud Deployment Topology

Hybrid Cloud Strategy

Running an 8-billion parameter LLM requires *\ge 6* GB of RAM, rendering free or entry-level cloud instances (e.g. AWS t2.micro with 1GB RAM) infeasible for hosting the LLM directly. FraudLens designs a realistic, production-viable hybrid topology:

```
[ Public Internet / Clients ]
              ■
              ▼
    ■■■■■■■■■■■■■■■■■■■■■■■■
    ■  Cloud Host       ■ (e.g. AWS Fargate, Render, Fly.io, or Railway)
    ■  • FastAPI (8000) ■ RAM: 512 MB - 1 GB
    ■  • LightGBM Model ■ CPU: 0.5 - 1 vCPU
    ■  • TreeSHAP       ■ Latency: &lt; 15ms
    ■■■■■■■■■■■■■■■■■■■■■■■■
              ■
              ■ (Secure VPN / mTLS / Private Enterprise Network)
              ▼
    ■■■■■■■■■■■■■■■■■■■■■■■■
    ■  On-Premise Host  ■ (Internal Banking Core / High-RAM Instance)
    ■  • Ollama Daemon  ■ RAM: 8 GB - 16 GB
    ■  • Llama 3.1 8B   ■ Zero Cloud PII Egress
    ■  • Private DB     ■ Regulatory Compliant
    ■■■■■■■■■■■■■■■■■■■■■■■■
```

PART VII: SETUP, EXECUTION & TROUBLESHOOTING GUIDE

FraudLens — Complete Setup, Execution & Troubleshooting Guide

Platform: Windows 10/11 (PowerShell) / Linux / macOS

Target Python Version: Python 3.13 (Default: `C:\Users\DHANRAJ\AppData\Local\Programs\Python\Python313\python.exe`)

Repository Path: `c:\Users\DHANRAJ\fraudlens`

---

1. System Requirements & Prerequisites

Resource	Minimum Specification	Recommended Specification
Operating System	Windows 10/11 64-bit, Ubuntu 22.04+, macOS	Windows 11 with PowerShell 7 or WSL2
Processor	4 Cores, x86_64 or ARM64	8 Cores (AVX2 support for LightGBM)
System Memory (RAM)	8 GB RAM (CPU-only Ollama)	16 GB RAM (for smooth concurrent LLM inference)
Storage	10 GB free SSD disk space	25 GB free SSD (if retaining full raw Kaggle CSVs)
Python	Python 3.13.x 64-bit	Python 3.13.9
Local LLM Runner	Ollama v0.3+	Ollama with CUDA / ROCm acceleration

---

2. Step-by-Step Installation & Setup

Step 1: Clone Repository

```
git clone https://github.com/DhanrajShitole/fraudlens.git
cd fraudlens
```

Step 2: Install Python Dependencies

Use the exact Python 3.13 executable path:

```
C:\Users\DHANRAJ\AppData\Local\Programs\Python\Python313\python.exe -m pip install --upgrade pip
C:\Users\DHANRAJ\AppData\Local\Programs\Python\Python313\python.exe -m pip install -r requirements.txt pytest reportlab
```

Step 3: Set Up Ollama for Local Agent Inference

1. Download and install Ollama from https://ollama.com.
2. Start the Ollama background daemon.
3. Pull the required 8-billion parameter model in PowerShell:

```
  ollama pull llama3.1:8b
```

4. Verify Ollama responds:

```
  ollama run llama3.1:8b &amp;quot;Hello&amp;quot;
```

---

3. Running the Test Suite

Execute the 51 automated unit, integration, drift, and model tests:

```
C:\Users\DHANRAJ\AppData\Local\Programs\Python\Python313\python.exe -m pytest tests/ -v
```

Expected Output:

```
tests/test_agent_logic.py::... PASSED (19 tests)
tests/test_api.py::... PASSED (12 tests)
tests/test_drift.py::... PASSED (6 tests)
tests/test_model.py::... PASSED (14 tests)
====================== 51 passed, 2 warnings in ~10.7s ======================
```

---

4. Running the FraudLens Application

Step 1: Start the FastAPI Backend

From the repository root (**`c:\Users\DHANRAJ\fraudlens`**):

```
C:\Users\DHANRAJ\AppData\Local\Programs\Python\Python313\python.exe -m uvicorn api.main:app --host 127.0.0.1 --port 8000 --reload
```

Verify startup in terminal:

```
Loading model and metadata...
Loaded model from MLflow registry (fraudlens-ieee-lightgbm@production).
Loading transactions for lookup (train + val combined, matching agent.py&#x27;s scope)...
Startup complete. model_source=registry:fraudlens-ieee-lightgbm@production
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

Step 2: Verify Backend Health

In another terminal:

```
curl http://127.0.0.1:8000/health
```

Expected output:

```
{&quot;status&quot;:&quot;ok&quot;,&quot;model_loaded&quot;:true,&quot;model_type&quot;:&quot;LightGBM (tuned, Phase 5 hyperparameters
```

Step 3: Launch the Analyst Dashboard

Double-click **`frontend/dashboard.html`** or open it in Google Chrome, Microsoft Edge, or Mozilla Firefox:

```
Start-Process &quot;frontend/dashboard.html&quot;
```

1. Verify the green banner shows: **`"Connected to FraudLens API."`**
2. Enter Transaction ID: **`3141064`** or **`3400379`**.
3. Press **Enter** or click **Get Score** to view fraud probability and SHAP reason codes.
4. Click **Investigate** to generate and display the evidence-grounded case file.

---

5. Running with Docker Compose (Local Container)

If Docker Desktop is installed on your machine:

```
# Build and run the scoring container
docker compose up --build -d

# Verify container health
curl http://localhost:8000/health

# Stop containers
docker compose down
```

Note on MLflow in Docker: **`mlflow.db`** stores host absolute paths. Inside Docker, the API logs a warning and transparently activates the **`joblib-fallback`** path, ensuring 100% availability.

---

6. Running Offline ML & MLOps Pipelines

Train Baseline Models (Logistic Regression, LightGBM, Isolation Forest)

```
C:\Users\DHANRAJ\AppData\Local\Programs\Python\Python313\python.exe src/train_baseline.py
```

Train Tuned Model with SMOTE (Phase 5)

```
C:\Users\DHANRAJ\AppData\Local\Programs\Python\Python313\python.exe src/train_advanced.py
```

Register Model in MLflow Registry & Assign `@production` Alias

```
C:\Users\DHANRAJ\AppData\Local\Programs\Python\Python313\python.exe src/register_model.py
```

Execute Feature Drift Detection (PSI Analysis)

```
C:\Users\DHANRAJ\AppData\Local\Programs\Python\Python313\python.exe src/drift_check.py
```

Run One-Shot Held-Out Test Evaluation (Generates Calibration & Test Reports)

```
C:\Users\DHANRAJ\AppData\Local\Programs\Python\Python313\python.exe src/evaluate_test.py
```

---

7. Common Issues & Troubleshooting

Issue 1: `HTTP 503: Agent investigation failed — the local Ollama service may have crashed or isn't running`

- **Cause:** Ollama background service is not running or the **`llama3.1:8b`** model has not been downloaded.
- **Resolution:**

1. Open a new terminal and run: **`ollama list`**.
2. If Ollama is stopped, start it from the Windows Start menu or run: **`ollama serve`**.
3. Ensure the model is pulled: **`ollama pull llama3.1:8b`**.
4. Test that it responds: **`ollama run llama3.1:8b "ping"`**.

Issue 2: `Port 8000 already in use`

- **Cause:** A previous Uvicorn process is still running in the background.
- **Resolution:**

```
  # Find process using port 8000
  netstat -ano | findstr :8000
  # Terminate process by PID (e.g. 12345)
  taskkill /PID 12345 /F
```

Issue 3: `FileNotFoundError: data/processed/ieee_train.parquet not found`

- **Cause:** Script was executed from inside `src/` rather than the repository root.
- **Resolution:** All FraudLens scripts dynamically resolve paths relative to the repository root. Always run commands from `c:\Users\DHANRAJ\fraudlens`.

Issue 4: `Warning: Could not load model from MLflow registry... Falling back to local joblib model`

- **Cause:** MLflow database cannot find the original artifact path (normal when moving directories or running inside Docker).
- **Resolution:** This is an intentional self-healing design feature. The API automatically falls back to `models/ieee_lightgbm_tuned.joblib` without any service interruption.

Issue 5: `SyntaxError: invalid character '—' (U+2014) in source code`

- **Cause:** Source files containing UTF-8 em-dashes were saved with a UTF-8 Byte Order Mark (BOM).
- **Resolution:** All source files in FraudLens are saved as standard UTF-8 without BOM. When editing, ensure your editor saves as UTF-8 (No BOM).

PART VIII: PRESENTATION, DEMO & VIVA VOCE DEFENSE GUIDE

FraudLens — Presentation, Viva Examination & Technical Interview Guide

Author: Dhanraj Shitole

Purpose: Comprehensive preparation material for project reviews, viva voce examinations, technical interviews, and industry presentations.

Part 1: Spoken Presentation Scripts

1. 30-Second Elevator Pitch

> *"Financial institutions lose billions to fraud, but the real operational bottleneck today is investigation throughput—compliance analysts spend up to 45 minutes manually assembling evidence for every flagged alert. I built **FraudLens**, an end-to-end platform combining cost-sensitive gradient boosted trees with a privacy-preserving local AI investigation copilot. It scores transactions in under 15 milliseconds with mathematical SHAP reason codes, and automatically generates audit-ready, evidence-backed case files under the rule: code makes decisions, models score, and AI writes prose. Tested on over 6.9 million transactions, FraudLens cuts analyst investigation time from minutes to seconds."*

2. 1-Minute Project Overview

> *"Hello everyone. In modern banking, fraud detection faces two major hurdles: extreme class imbalance—where fraud accounts for under 1% of transactions—and analyst alert fatigue. Traditional rules generate too many false alarms, while naive LLMs hallucinate facts and leak future data.*
>
> *To solve this, I developed **FraudLens**. On the predictive side, FraudLens uses a tuned LightGBM model trained on leakage-safe temporal splits of the IEEE-CIS dataset, achieving a PR-AUC of 0.598—a 7.3% improvement over untuned baselines—and an ROC-AUC of 0.924. Crucially, we calibrate our decision threshold against a strict 2% false-positive budget to control operational costs, and calculate top-5 TreeSHAP reason codes for instant transparency.*
>
> *On the investigation side, FraudLens introduces a tool-scoped local AI agent powered by Llama 3.1 8B. Instead of letting the LLM guess, deterministic Python tools extract point-in-time entity velocity and check enterprise fraud policies with a relevance floor. An automated validator enforces that every number is grounded in evidence before an analyst sees it. The system is fully containerized, backed by an MLflow registry and Population Stability Index drift tracking, and validated with a 51-test automated suite."*

3. 5-Minute Academic / Viva Presentation Script

> **"Good morning respected evaluators. Today I am presenting my final-year project, **FraudLens: A Real-Time, Explainable Financial Fraud Detection Platform with an Evidence-Grounded Agentic Investigation Copilot.***
>
> ***1. The Problem:***
>
> *When machine learning models flag suspicious transactions, compliance officers must cross-reference account histories, device velocity, and organizational rules to decide whether to freeze the account. This manual context-assembly takes 15 to 45 minutes per alert.*
>
> ***2. The Data & Leakage Prevention:***

We evaluated FraudLens on two major benchmarks: the real-world IEEE-CIS dataset (590,000 transactions) and synthetic PaySim (6.3 million transactions). We strictly rejected random train-test splitting because it causes severe lookahead leakage in temporal transaction streams. Instead, we implemented chronological 70/15/15 partitioning: training on the earliest 70%, validating on the middle 15%, and evaluating one-shot generalization on the final held-out 15%.

3. Feature Engineering & The Anomaly Detection Fallacy:

We engineered 545 candidate features. Crucially, because device IDs were 80% missing, we built composite entity aggregations on the tuple **`(card1, addr1)`***—the physical card issuer and billing district. Using TreeSHAP, we proved these entity features rank in the top 7% of all 545 features. On PaySim, we tested the common hypothesis that unsupervised anomaly detection outperforms supervised models under extreme 1,200:1 imbalance. Our results disproved this: supervised LightGBM achieved 94.5% recall at a 2% FP budget versus only 2.5% for Isolation Forest, because supervised trees efficiently learn discrete account-draining logic.*

4. Model Evaluation & Calibration:

On IEEE-CIS, our tuned LightGBM model with SMOTE achieved a validation PR-AUC of 0.5975, an ROC-AUC of 0.9242, and an F1 score of 0.5405 at a 2% false-positive budget threshold of \tau = 0.129365. On the held-out test set, it retained a 15.3x lift over base fraud prevalence and achieved a Brier Skill Score of 34.12%, proving solid probability calibration.

5. The Grounded Investigation Copilot:

To eliminate analyst investigation bottlenecks, we built an agentic copilot. In early testing, we discovered 4 distinct hallucination classes in standard LLMs: geographic errors, data-granularity bugs, label leakage, and vacuous self-referential comparisons. We resolved this through the architectural principle: 'Code makes decisions, models score, and LLMs narrate.' The agent executes deterministic, point-in-time Python tools first, builds a policy search query from the evidence, and runs TF-IDF with a relevance floor. Llama 3.1 8B running locally via Ollama drafts two narrative lines, which are passed through an automated regex and numerical validator before being merged into the case file. If Ollama crashes or validation fails, a fallback engine outputs a 100% accurate tool-derived case file with zero data loss.

6. Production Architecture & MLOps:

FraudLens is served via a FastAPI backend delivering sub-15ms scoring, persistent SQLite case logging, a responsive React analyst dashboard, MLflow model registry aliasing (`@production`*), Population Stability Index (PSI) drift monitoring across all 545 features, and a 51-test automated CI/CD pipeline in GitHub Actions.*

*Thank you, and I am now ready for your questions."**

---

4. Step-by-Step Live Demonstration Guide

1. System Health Check:

- Open browser or terminal to **`http://127.0.0.1:8000/health`**. Show that **`model_loaded: true`** and **`model_source`** points to **`registry:fraudlens-ieee-lightgbm@production`** (or **`joblib-fallback`**).

2. Scoring a Normal Transaction:

- Open **`frontend/dashboard.html`**.
- Enter a known legitimate Transaction ID (e.g. **`2987000`**).
- Click **Get Score** (or press Enter).
- Show the low probability (< 5%), green **`BELOW THRESHOLD`** badge, and explain that the threshold is calibrated to a 2% false-positive budget (*\tau = 0.129365*).
- Point out the SHAP reason bars showing which features reduced risk.

3. Scoring a Flagged Fraud Transaction:

- Enter a known flagged Transaction ID (e.g. **`3141064`** or **`3400379`**).
- Click **Get Score**.

- Show the high probability (> 80%), red `FLAGGED FOR REVIEW` badge, and highlight the top SHAP contributor: `card1_addr1_amt_mean` pushing risk up.

4. Triggering the Investigation Copilot:

- With the flagged transaction loaded, click the gold **Investigate** button.
- Watch the investigation assemble. Show the 5 structured sections:
- `ENTITY SUMMARY`: Total transactions and dollar spend.
- `TOP RISK FACTORS`: Bulleted grounded facts.
- `RELATED ENTITY SIGNAL`: Multi-domain velocity across card+address.
- `RELEVANT POLICY`: Matched enterprise policy clause (e.g. `POL-003: Email Domain Fraud Ring`).
- `RECOMMENDED ACTION`: Code-generated escalation recommendation.

5. Demonstrating Audit History:

- Point to the sidebar under **Recent Investigations**. Show the newly created case.
- Click the case to show instant retrieval from SQLite (`cases.db`).

Part 2: Comprehensive Viva & Technical Interview Question Bank (100+ Questions)

Category A: General Project & System Architecture

Q1: What is the core problem FraudLens solves?

Answer: FraudLens addresses the dual challenge of high-throughput automated fraud scoring under severe class imbalance and eliminating the 15-to-45-minute manual evidence-gathering bottleneck faced by compliance officers reviewing flagged transactions.

Q2: What is the high-level architecture of FraudLens?

Answer: A decoupled hybrid pattern consisting of an offline training/MLOps pipeline (LightGBM, SMOTE, MLflow, PSI drift check) and an online serving stack (FastAPI backend, TreeSHAP reason codes, read-only deterministic tools, local Llama 3.1 8B copilot, SQLite case persistence, and a single-file React analyst dashboard).

Q3: What makes FraudLens different from standard AI fraud detection tools?

Answer: Standard tools either rely solely on black-box probabilities or use unconstrained generative chatbots that hallucinate facts. FraudLens enforces the principle that *code makes decisions, models score, and LLMs write prose*. The model cannot alter the fraud verdict, and all numerical claims are validated against point-in-time evidence.

Q4: Who are the primary end users of FraudLens?

Answer: Anti-Money Laundering (AML) analysts, Level-1 and Level-2 fraud investigation officers, and compliance risk teams in financial institutions and payment gateways.

Q5: Why did you not build a live transaction ingestion stream with Kafka?

Answer: In our BTech academic scope, historical parquet files simulate the lookup storage layer. In a production enterprise setting, Kafka would stream transaction payloads to the `/score` endpoint, while historical feature aggregates would reside in an operational feature store (e.g. Feast or Redis).

Category B: AI & Machine Learning Fundamentals

Q6: What type of machine learning task is fraud detection?

Answer: Supervised binary classification under extreme class imbalance.

Q7: What is the target variable in IEEE-CIS and what do values mean?

Answer: `isFraud`, where `1` denotes confirmed fraudulent transactions (chargebacks, compromised cards) and `0` denotes legitimate transactions.

Q8: Why is standard classification accuracy a useless metric in fraud detection?

Answer: Because of class imbalance. In a dataset with 1% fraud, a naive classifier predicting 0 for every transaction achieves 99% accuracy while detecting zero fraud.

Q9: What is the bias-variance tradeoff in gradient boosting?

Answer: Bias represents error from overly simplistic assumptions (underfitting); variance represents sensitivity to small fluctuations in training data (overfitting). LightGBM controls variance through regularization parameters (`num_leaves`, `min_child_samples`, `colsample_bytree`, and `subsample`).

Q10: What is data leakage, and how did FraudLens prevent it?

Answer: Data leakage occurs when information from outside the training partition influences the model. We prevented it by enforcing chronological 70/15/15 time splits and fitting all encoders, imputers, and SMOTE resamplers strictly on training data.

Category C: Datasets, Preprocessing & Feature Engineering

Q11: Why is random k-fold cross-validation flawed for fraud detection?

Answer: Random splits allow transactions from the same card/entity across a single fraud attack to be split between train and test. The model memorizes entity identifiers and predicts past fraud using future records, inflating validation scores artificially.

Q12: How did you handle missing values in IEEE-CIS?

Answer: Categorical missing values were assigned a `'missing'` string token. Numerical missing values were imputed with `-999`, accompanied by binary missingness indicator columns. Tree algorithms naturally isolate `-999` to specific branch routes without distorting distributions.

Q13: How did you handle high-cardinality categorical variables?

Answer: Frequency encoding fit strictly on the training partition. Each category is replaced by its normalized frequency in training data, avoiding the curse of dimensionality caused by one-hot encoding.

Q14: What composite entity was engineered, and why?

Answer: The tuple `(card1, addr1)` representing card issuer bank and geographic billing district. Device and IP columns had >80% missingness, whereas `(card1, addr1)` was available on almost all transactions and effectively captured entity-level behavioral clusters.

Q15: What point-in-time statistics were computed for this entity?

Answer: Historical transaction count (`card1_addr1_count`), mean spend (`card1_addr1_amt_mean`), spend standard deviation (`card1_addr1_amt_std`), and distinct email count (`card1_addr1_email_count`).

Category D: LightGBM, Gradient Boosting & Tree Algorithms

Q16: How does gradient boosting work?

Answer: It builds an additive ensemble of weak decision trees sequentially. Each new tree fits to the pseudo-residuals (negative gradient of the loss function) of the existing ensemble, progressively minimizing overall loss.

Q17: Why did you choose LightGBM over XGBoost and Random Forest?

Answer: Random Forest does not optimize sequential residuals and handles imbalance poorly. XGBoost uses exact greedy split-finding or pre-sorted algorithms that are slow on large datasets. LightGBM uses histogram-based binning (256 discrete bins) and leaf-wise tree growth, reducing training memory by over 70% and accelerating training by 5x–10x.

Q18: What is leaf-wise tree growth vs level-wise growth?

Answer: Level-wise (XGBoost) grows all nodes in a tree level symmetrically. Leaf-wise (LightGBM) splits only the single leaf with the maximum delta loss reduction, achieving lower error for a fixed number of leaves.

Q19: How does LightGBM handle numerical missing values natively?

Answer: During split evaluation, missing values are placed entirely into the left child, then the right child; the direction yielding higher gain is saved as the default route for unseen missing instances.

Q20: What is `scale_pos_weight` in LightGBM?

Answer: A multiplier applied to the gradient of the positive class. Setting `scale_pos_weight = N_neg / N_pos` forces the tree optimizer to treat false negatives as severely as false positives.

Category E: Imbalanced Learning & SMOTE

Q21: What is SMOTE and how does it generate synthetic samples?

Answer: Synthetic Minority Over-sampling Technique. For each minority instance, it finds its k-nearest minority neighbors in feature space and interpolates a synthetic point along the line segment connecting them: $x_{\text{new}} = x_i + \lambda(x_{zi} - x_i)$ where $\lambda \sim U(0, 1)$.

Q22: Why did you apply SMOTE with `sampling_strategy = 0.1` instead of `1.0`?

Answer: Resampling the minority class to 50/50 balance (1.0) severely distorts feature space geometry and leads to high false positives. Balancing to 1:10 (0.1) provides sufficient minority density for gradient boosting without flooding the dataset with synthetic artifacts.

Q23: How did you prevent SMOTE from leaking data into validation?

Answer: SMOTE was applied exclusively to the training split *after* temporal partitioning. The validation and held-out test sets remained completely untouched with their natural 27.4:1 class distribution.

Q24: What are the risks of using SMOTE in high-dimensional tabular data?

Answer: When features are sparse or collinear, linear interpolation can create synthetic samples in empty regions of feature space that do not correspond to realistic transactions. Combining SMOTE with leaf-wise gradient boosting and tree pruning mitigates this.

Category F: Model Evaluation Metrics & The 2% False-Positive Budget

Q25: Why is PR-AUC preferred over ROC-AUC in fraud detection?

Answer: ROC-AUC plots True Positive Rate vs False Positive Rate ($\text{FP} / \text{All Legit}$). Because legitimate transactions dominate by 27:1 or 1,200:1, thousands of false positives barely move the false-positive rate, making ROC-AUC look deceptively optimistic (>0.92). PR-AUC plots Precision ($\text{TP} / [\text{TP} + \text{FP}]$) vs Recall; any influx of false positives directly crushes Precision.

Q26: What was your model's validation PR-AUC, and what was the lift over baseline?

Answer: The tuned model achieved a validation PR-AUC of **0.5975**, a **+7.3% relative lift** over untuned LightGBM (0.5566) and nearly double Logistic Regression (0.3050).

Q27: What is the 2% false-positive budget, and why did you choose it?

Answer: In real banking compliance, human review teams have fixed staffing capacity. Capping the false-positive rate at 2.0% guarantees that no more than 2% of legitimate transactions trigger manual alerts. The operational threshold $\tau = 0.129365$ was chosen at the 98th percentile of legitimate validation scores.

Q28: What were the test set results at this threshold on IEEE-CIS?

Answer: On the held-out test set ($N=88,581$), the model achieved 53.39% Recall, 43.95% Precision, an F1 score of 0.4821, and an actual false-positive rate of 2.46%.

Q29: What is a Brier Score and Brier Skill Score?

Answer: The Brier score measures mean squared probability error: $BS = \frac{1}{N} \sum (\hat{p}_i - y_i)^2$. On test, our model achieved $BS = 0.02213$ vs 0.03359 for a non-informative constant base rate, yielding a Brier Skill Score of **34.12%**, confirming strong probabilistic calibration.

Category G: TreeSHAP & Explainable AI

Q30: What is the difference between global and local explainability?

Answer: Global explainability measures overall feature importance across the entire dataset (e.g. mean $|SHAP|$). Local explainability explains the specific prediction for a single transaction by showing exactly how much each feature shifted the score above or below the base rate.

Q31: What are Shapley values from game theory?

Answer: Shapley values allocate the payout of a cooperative game fairly among players based on their marginal contributions across all possible player coalitions. In machine learning, the players are features, and the payout is the model prediction.

Q32: Why use TreeSHAP instead of KernelSHAP or LIME?

Answer: KernelSHAP and LIME rely on heuristic sampling and take exponential time *O(2^d)* or minutes per sample. TreeSHAP exploits tree graph structures to compute exact Shapley values in polynomial time *O(T L D^2)*, completing local explanations in milliseconds.

Q33: How does the dashboard visualize SHAP values?

Answer: The top 5 features by absolute magnitude *|\phi_i|* are displayed as horizontal bars: red bars indicate positive contributions pushing risk up; green bars indicate risk-reducing contributions.

Category H: Agentic AI & Investigation Copilot

Q34: What is the difference between a chatbot and an AI agent?

Answer: A chatbot generates text based solely on user prompts. An AI agent is tool-scoped: it observes an environment, executes read/write tools, inspects deterministic state, and synthesizes structured actions.

Q35: Why should an LLM never decide the fraud verdict?

Answer: Because LLMs are non-deterministic, probabilistic text generators. Giving an LLM decision authority over banking blocks introduces non-reproducible compliance decisions, hallucinated justifications, and legal liabilities.

Q36: What model powers the agent, and why was it selected?

Answer: Llama 3.1 8B running locally through Ollama. It was selected because it runs completely offline without sending customer PII to cloud APIs, incurs zero per-token API costs, and supports concise structured prompting.

Q37: What was the geographic hallucination bug in early testing?

Answer: The model misconstrued **`addr1 = 299`** as an international country code and fabricated a cross-border fraud justification. It was fixed with explicit prompt constraints explaining **`addr1`** is an internal regional billing code.

Q38: How does the agent prevent temporal label leakage?

Answer: In **`src/agent.py`**, all evidence tool queries strictly filter to **`TransactionDT <= this transaction`**. Prior confirmed fraud counts strictly require **`TransactionDT < this transaction`** and **`TransactionID != this transaction`**.

Q39: What is the vacuous comparative framing bug, and how is it blocked?

Answer: The model wrote statements like *"mean transaction amount of 144.14 is greater than the overall mean of 144.14"*. The hardened validator uses regex **`_COMPARATIVE_RE`** to reject comparative phrasing unless the sentence contains at least two distinct grounded numbers.

Q40: What happens if Ollama crashes during an investigation?

Answer: The backend catches **`_LLMError`** and immediately triggers **`_fallback_case_file()`**, assembling all tool evidence and policy citations into the standard 5-section case file with 0% data loss.

Category I: Backend, API & Database

Q41: Why was FastAPI chosen over Flask or Django?

Answer: FastAPI provides native async ASGI concurrency, automatic Pydantic data validation, sub-15ms response times, and automated interactive OpenAPI/Swagger documentation at **`/docs`**.

Q42: What endpoints does the backend expose?

Answer: **`GET /health`**, **`POST /score/{transaction_id}`**, **`POST /investigate/{transaction_id}`**, **`GET /cases`**, and **`GET /cases/{case_id}`**.

Q43: How does the backend store investigation cases?

Answer: Cases are logged to a local SQLite database (`cases.db`) in table `cases` containing `id`, `transaction_id`, `risk_score`, `threshold_used`, `flagged`, `top_reason_codes` (JSON), `case_file`, and UTC `created_at`.

Q44: Why use SQLite instead of PostgreSQL for this phase?

Answer: SQLite is serverless, zero-configuration, self-contained in a single file, and easily reproducible across academic evaluation environments. Migrating to PostgreSQL is a seamless next step for multi-analyst cloud deployment.

Category J: MLOps, Drift Detection & DevOps

Q45: What is the Population Stability Index (PSI)?

Answer: A statistical metric measuring the divergence between a reference distribution P and comparison distribution Q: $\text{PSI} = \sum (Q_b - P_b) \ln(Q_b / P_b)$. $\text{PSI} < 0.1$ indicates stability.

Q46: Why did `TransactionDT` exhibit a PSI of 8.28, and why is that not real drift?

Answer: `TransactionDT` is the chronological time variable used to split train (earliest 70%) from validation (next 15%). The two distributions have completely disjoint supports; it is a split artifact, not behavioral drift.

Q47: Which features exhibited genuine drift in your analysis?

Answer: Exactly 7 features (1.3% of feature space), primarily device and network fingerprint headers (`id_13`, `id_20`, `id_02`) reflecting evolving operating system versions.

Q48: How does MLflow handle model registry promotion?

Answer: We use MLflow's modern Model Aliases API (`client.set_registered_model_alias`), attaching the `@production` alias to the active version of `fraudlens-ieee-lightgbm`.

Q49: What checks are performed in GitHub Actions CI?

Answer: Python 3.13 dependency installation, bytecode compilation syntax check (`compileall`), HTML syntax parsing of the dashboard, automated pytest execution of 51 tests, and Docker container build verification.

Q50: Why can the LLM agent not easily run on free cloud instances?

Answer: Llama 3.1 8B requires ≥ 6 GB of RAM for model weights and KV cache. Free cloud tiers typically provide 512MB to 1GB RAM. FraudLens resolves this with a hybrid architecture: scoring runs in lightweight containers, while the LLM runs locally or on a private high-RAM server.

PART IX: MASTER 20-CHAPTER LEARNING HANDBOOK

Project Title: FraudLens — A Real-Time, Explainable Financial Fraud Detection Platform with an Evidence-Grounded Agentic Investigation Copilot

Author & Developer: Dhanraj Shitole

Degree / Academic Focus: Bachelor of Technology (BTech) in Computer Science & Engineering

Version: 1.0 (Comprehensive Master Edition)

Date of Release: November 2026

Repository: https://github.com/DhanrajShitole/fraudlens

Master Table of Contents

- [Executive Overview & Supporting Guides](#executive-overview--supporting-guides)
- [Chapter 1: Project Introduction & Problem Definition](#chapter-1-project-introduction--problem-definition)
- [Chapter 2: Complete System Architecture & Component Interactions](#chapter-2-complete-system-architecture--component-interactions)
- [Chapter 3: Dataset Analysis & Leakage-Safe Data Engineering](#chapter-3-dataset-analysis--leakage-safe-data-engineering)
- [Chapter 4: Comprehensive AI, Machine Learning & Explainability Theory](#chapter-4-comprehensive-ai-machine-learning--explainability-theory)
- [Chapter 5: Evidence-Grounded Agentic AI Investigation Copilot](#chapter-5-evidence-grounded-agentic-ai-investigation-copilot)
- [Chapter 6: Backend Engineering & REST API Documentation](#chapter-6-backend-engineering--rest-api-documentation)
- [Chapter 7: Frontend Architecture & Analyst Dashboard](#chapter-7-frontend-architecture--analyst-dashboard)
- [Chapter 8: Database Architecture & Case Persistence](#chapter-8-database-architecture--case-persistence)
- [Chapter 9: MLOps: Model Registry, Promotion & Population Stability Index Drift](#chapter-9-mlops-model-registry-promotion--population-stability-index-drift)
- [Chapter 10: DevOps, Containerization & CI/CD Automation](#chapter-10-devops-containerization--cicd-automation)
- [Chapter 11: Testing, Quality Assurance & Verification Suite](#chapter-11-testing-quality-assurance--verification-suite)
- [Chapter 12: Technology Stack & Dependency Inventory](#chapter-12-technology-stack--dependency-inventory)
- [Chapter 13: Complete End-to-End Execution Guide](#chapter-13-complete-end-to-end-execution-guide)
- [Chapter 14: File-by-File Codebase Walkthrough](#chapter-14-file-by-file-codebase-walkthrough)
- [Chapter 15: Technical Decision Justifications & Architecture Trade-Offs](#chapter-15-technical-decision-justifications--architecture-trade-offs)
- [Chapter 16: System Limitations, Risks & Realistic Future Scope](#chapter-16-system-limitations-risks--realistic-future-scope)
- [Chapter 17: Complete Presentation, Pitch & Demonstration Guide](#chapter-17-complete-presentation-pitch--demonstration-guide)
- [Chapter 18: Viva Voce, Technical Interview & Industry Question Bank (100+ Questions)](#chapter-18-viva-voce-technical-interview--industry-question-bank-100-questions)
- [Chapter 19: Comprehensive Glossary of Technical Terms](#chapter-19-comprehensive-glossary-of-technical-terms)
- [Chapter 20: Self-Assessment Knowledge & Readiness Checklist](#chapter-20-self-assessment-knowledge--readiness-checklist)

Executive Overview & Supporting Guides

This handbook serves as the definitive reference manual for the **FraudLens** platform. For deeper exploration of specific subsystems, refer to the dedicated companion guides in the `docs/` directory:

1. [`docs/FRAUDLENS_EXECUTIVE_SUMMARY.md`](FRAUDLENS_EXECUTIVE_SUMMARY.md): Executive summary for recruiters, evaluators, and stakeholders.
2. [`docs/FRAUDLENS_TECHNICAL_ARCHITECTURE.md`](FRAUDLENS_TECHNICAL_ARCHITECTURE.md): Component diagrams, sequence charts, and interaction protocols.
3. [`docs/FRAUDLENS_API_REFERENCE.md`](FRAUDLENS_API_REFERENCE.md): Detailed REST API specification, schemas, and cURL examples.
4. [`docs/FRAUDLENS_ML_DEEP_DIVE.md`](FRAUDLENS_ML_DEEP_DIVE.md): Mathematical formulations of LightGBM, SMOTE, TreeSHAP, and held-out test evaluations.
5. [`docs/FRAUDLENS_AI_AGENT_DEEP_DIVE.md`](FRAUDLENS_AI_AGENT_DEEP_DIVE.md): Agentic copilot mechanics, hallucination mitigation, and automated validators.
6. [`docs/FRAUDLENS_DEVOPS_MLOPS_GUIDE.md`](FRAUDLENS_DEVOPS_MLOPS_GUIDE.md): MLflow registry aliasing, PSI drift mathematics, Docker, and CI/CD pipelines.
7. [`docs/FRAUDLENS_PRESENTATION_AND_VIVA_GUIDE.md`](FRAUDLENS_PRESENTATION_AND_VIVA_GUIDE.md): Spoken presentation scripts, demonstration flows, and 100+ viva questions.
8. [`docs/FRAUDLENS_SETUP_AND_TROUBLESHOOTING.md`](FRAUDLENS_SETUP_AND_TROUBLESHOOTING.md): Step-by-step Windows setup, execution commands, and troubleshooting FAQ.

Chapter 1: Project Introduction & Problem Definition

1.1 What is FraudLens?

FraudLens is a full-stack, real-time, explainable financial fraud detection system integrated with an evidence-grounded agentic investigation copilot. It combines cost-sensitive machine learning (LightGBM tuned with SMOTE) to predict transaction fraud probabilities in milliseconds with a privacy-preserving, local AI investigation agent (Llama 3.1 8B via Ollama).

1.2 The Problem: The Investigation Throughput Bottleneck

In modern commercial banking, fraud detection faces two asymmetric bottlenecks:

1. **Extreme Class Imbalance:** Fraudulent transactions represent under 3.5% in typical card streams and under 0.15% in mobile money flows. Standard classifiers trained on accuracy default to predicting "legitimate" for every transaction, failing to intercept fraudulent losses.
2. **The Investigation Throughput Bottleneck:** Detection models flag alerts, but human compliance analysts spend 15 to 45 minutes per alert manually assembling context: cross-referencing past card transactions, evaluating IP clusters, checking regulatory rules, and writing narrative case files.

Generic Generative AI cannot be deployed naively into this workflow because unconstrained models hallucinate numbers, invent non-existent cross-border rules, leak future labels, and generate vacuous tautologies.

1.3 Core Innovation & Architectural Rule

FraudLens enforces a strict architectural boundary:

> ***"Code makes decisions, models score, and language models write prose."***

The machine learning model computes fraud risk; deterministic Python code queries point-in-time entity histories and matches organizational policies; and a local LLM is restricted to writing two narrative summary lines that are mathematically validated against tool evidence before any human investigator sees them.

1.4 How to Pitch FraudLens in Interviews and Reviews

A. The 30-Second Elevator Pitch

> *"Financial institutions lose billions annually to transaction fraud, but the real operational bottleneck today is investigation throughput—compliance officers spend up to 45 minutes manually assembling evidence for every flagged alert. I built FraudLens, an end-to-end platform combining cost-sensitive gradient boosted trees with a privacy-preserving local AI investigation copilot. It scores transactions in under 15 milliseconds with mathematical SHAP reason codes, and automatically generates audit-ready, evidence-backed case files under the rule: code makes decisions, models score, and AI writes prose. Tested on over 6.9 million transactions, FraudLens cuts analyst investigation time from minutes to seconds."*

B. The 1-Minute Pitch

> *"In modern banking, fraud detection systems suffer from alert fatigue: traditional rules flag too many false alarms, while black-box models provide no transparency. When analysts investigate, assembling context takes dozens of minutes per case.*
>
> *I designed FraudLens to solve both problems. On the detection side, FraudLens uses a tuned LightGBM model trained on leakage-safe temporal splits of the IEEE-CIS dataset, achieving an ROC-AUC of 0.924 and a PR-AUC of 0.598—a 7.3% improvement over untuned baselines. We calibrate our decision threshold to a strict 2% false-positive budget to control operational costs, and calculate top-5 TreeSHAP reason codes for instant transparency.*
>
> *On the investigation side, FraudLens introduces a tool-scoped local AI agent powered by Llama 3.1 8B. Instead of letting the LLM guess, deterministic Python tools extract point-in-time entity velocity and check enterprise fraud policies. An automated validator enforces that every number is grounded in evidence before an analyst sees it. The system is fully containerized, backed by an MLflow registry and Population Stability Index drift tracking, and validated with a 51-test automated suite."*

C. The 3-Minute Comprehensive Presentation

> *"Good morning. Today I am presenting FraudLens, an end-to-end platform bridging high-speed predictive fraud detection and evidence-based AI copilot investigations.*
>
> *First, let's look at the machine learning challenge. Evaluating models using random k-fold cross-validation on financial transactions creates severe lookahead leakage, where future fraud patterns from the same card are used to predict past transactions. In FraudLens, we enforce strict chronological 70/15/15 partitioning across 590,000 IEEE-CIS transactions and 6.3 million PaySim transactions. We engineered 545 modeling features, including composite entity aggregations on card and billing address tuples that rank in the top 7% of SHAP feature importance. On PaySim, we tested the common hypothesis that unsupervised anomaly detection outperforms supervised models under extreme 1,200:1 imbalance; our empirical results proved the opposite: supervised LightGBM achieved 94.5% recall at a 2% FP budget versus only 2.5% for Isolation Forest, because supervised trees efficiently learn discrete account-draining logic.*
>
> *Second, we tuned LightGBM with SMOTE resampling on IEEE-CIS, achieving a validation PR-AUC of 0.598 and an ROC-AUC of 0.924. On the final held-out test set, evaluated strictly one-shot with frozen parameters, the model retained a 15.30x lift over the base fraud rate and achieved a Brier Skill Score of 34.12%, confirming strong probability calibration.*
>
> *Third, to eliminate the manual compliance bottleneck, we built an AI investigation copilot. Early testing revealed that unconstrained 8B LLMs suffer from 4 hallucination classes: geographic errors, data-granularity bugs, temporal label leakage, and vacuous self-referential comparisons. We redesigned the architecture so that deterministic Python code gathers point-in-time evidence and queries policy clauses via TF-IDF with a relevance floor. Llama 3.1 8B running locally via Ollama drafts two narrative sentences, which are passed through an automated regex and numerical validator before being merged into the case file. If Ollama crashes or validation fails, a fallback engine outputs a 100% accurate tool-derived case file with zero data loss.*
>
> *Finally, FraudLens includes a FastAPI backend with sub-15ms scoring, persistent SQLite case logging, a responsive React analyst dashboard, MLflow model registry aliasing (`@production`), Population Stability Index (PSI) drift monitoring across all 545 features, and a 51-test automated CI/CD pipeline in GitHub Actions."*

Chapter 2: Complete System Architecture & Component Interactions

2.1 The End-to-End Architectural Flow

FraudLens operates across two distinct operational pipelines: an **Offline ML Training & MLOps Pipeline** and an **Online Serving & Investigation Stack**.

```
graph TD
    subgraph Data Layer
        D1[IEEE-CIS Dataset 590K] --&gt; DP[src/features.py]
        D2[PaySim Dataset 6.3M] --&gt; DP
        DP --&gt; P1[data/processed/ieee_train.parquet]
        DP --&gt; P2[data/processed/ieee_val.parquet]
        DP --&gt; P3[data/processed/ieee_test.parquet]
    end

    subgraph Offline MLOps Pipeline
        P1 --&gt; TR[src/train_advanced.py]
        P2 --&gt; TR
        TR --&gt; M1[models/ieee_lightgbm_tuned.joblib]
        TR --&gt; M2[models/ieee_lightgbm_tuned_metadata.json]
        TR --&gt; MLF[(MLflow SQLite: mlflow.db)]
        M1 --&gt; REG[src/register_model.py]
        REG --&gt; REG_ALIAS[Model Registry: @production]
        P1 --&gt; DRIFT[src/drift_check.py]
        P2 --&gt; DRIFT
        DRIFT --&gt; DR_REP[reports/drift_report_top20.png]
    end

    subgraph Serving &amp;amp; API Layer
        M1 -.-&gt;|Fallback Path| API[api/main.py FastAPI]
        REG_ALIAS -.-&gt;|Primary Path| API
        API --&gt; DB[(SQLite: cases.db)]
    end

    subgraph User &amp;amp; Investigation Layer
        UI[frontend/dashboard.html] &lt;--&gt;|HTTP / REST| API
        API --&gt; SC[Scoring &amp;amp; SHAP Engine]
        API --&gt; AG[src/agent.py Investigation Copilot]
        AG --&gt; OLLAMA[Ollama: llama3.1:8b]
        AG --&gt; VAL[Hardened Automated Validator]
        VAL --&gt; CASE[Structured Case File]
        CASE --&gt; DB
        CASE --&gt; UI
    end
```

2.2 Execution Sequence: Real-Time Scoring Flow

When an analyst enters a Transaction ID in **`frontend/dashboard.html`**:

1. The frontend initiates **`POST /score/{transaction_id}`** to FastAPI (**`api/main.py`**).
2. The backend looks up the transaction row in the loaded dataset store (**`train_data`** + **`val_data`**).
3. If not found, HTTP 404 is returned.
4. If found, features are extracted, missing values filled with **`-999`**, and passed to LightGBM.
5. LightGBM returns the calibrated probability $P(\text{isFraud} = 1 \mid X)$.
6. TreeSHAP decomposes the prediction into feature attributions, sorting the top 5 absolute contributors.
7. Risk score is compared against the 2% FP budget threshold ($\tau = 0.129365$).
8. A JSON response (**`ScoreResponse`**) is returned, and the UI displays the risk percentage, badge, and reason bars.

Chapter 3: Dataset Analysis & Leakage-Safe Data Engineering

3.1 Dataset Inventory & Imbalance Comparison

Parameter	IEEE-CIS Fraud Detection	PaySim Mobile Money
Source & Type	Vesta Corporation / Kaggle (Real e-commerce)	Synthetic multi-agent financial simulator
Total Observations	590,540 transactions	6,362,620 transactions
Confirmed Frauds	20,663 transactions	8,213 transactions
Class Imbalance	**27.4 : 1** (3.50% fraud prevalence)	**1,221.6 : 1** (0.13% fraud prevalence)
Chronological Split	70% Train (413,378), 15% Val (88,581), 15% Test (88,581)	70% Train (4,453,834), 15% Val (954,393), 15% Test (954,393)

3.2 Key Verified Features & Engineering Table

Feature Name	Type	Origin	Meaning & Fraud Significance
`TransactionAmt`	Float	Original	Transaction amount in USD. Frauds often spike in specific dollar brackets.
`TransactionDT`	Int	Original	Elapsed seconds from reference date. Transformed into cyclical hour and day.
`card1`	Int	Original	Payment card issuer bank identifier. Core component of composite entity.
`addr1`	Int	Original	Regional billing district code (NOT country). Grouped with `card1`.
`P_emaildomain`	String	Original	Purchaser email domain (e.g. gmail.com, anonymous protonmail).
`card1_addr1_count`	Int	Engineered	Number of cumulative point-in-time transactions on this card+address. (SHAP Rank 17).
`card1_addr1_amt_mean`	Float	Engineered	Running mean spend for this entity. Flags abnormal charge sizes. (SHAP Rank 13).
`card1_addr1_amt_std`	Float	Engineered	Spend volatility for entity. Sudden deviations indicate card compromise. (SHAP Rank 24).
`card1_addr1_email_count`	Int	Engineered	Distinct email domains associated with entity. High count signals fraud rings. (SHAP Rank 39).
`hour_of_day`	Int	Engineered	Hour (0–23) extracted from `TransactionDT`. Captures overnight attack velocity.
`day_of_week`	Int	Engineered	Day (0–6) extracted from `TransactionDT`. Captures weekend fraud patterns.

Chapter 4: Comprehensive AI, Machine Learning & Explainability Theory

4.1 Ten-Point Teaching: Gradient-Boosted Decision Trees & LightGBM

1. What is it?

LightGBM (Light Gradient Boosting Machine) is an ensemble machine learning algorithm that builds a collection of decision trees sequentially, optimizing a binary cross-entropy loss function.

2. Why is it needed?

Financial tabular data contains non-linear interactions, skewed amounts, and high missingness that linear models fail to capture. Standard deep learning underperforms on tabular data, while XGBoost is memory-intensive on 500K+ rows.

3. How does it work in general?

It trains trees iteratively. Each tree predicts the negative gradient (pseudo-residuals) of the loss function with respect to the prior ensemble output. Continuous features are discretized into 256 histogram bins.

4. How is it implemented in FraudLens?

In `src/train_advanced.py` and served in `api/main.py`. Configured with `LGBMClassifier(num_leaves=63, learning_rate=0.03, n_estimators=500, colsample_bytree=0.8, subsample=0.8)`.

5. Which file implements it?

`src/train_advanced.py` (training), `models/ieee_lightgbm_tuned.joblib` (serialized weights), and `api/main.py` (serving).

6. Step-by-Step Execution:

X feature matrix is loaded \to `-999` fills missing values \to histogram binning \to sequential leaf-wise tree construction \to sigmoid transformation yields final probability *hat{p}(x)*.

7. Real-World Example:

A transaction of *450 on a card that usually spends* \20 at 3 AM routes through split nodes testing `card1_addr1_amt_mean` and `hour_of_day`, landing in a terminal leaf with high log-odds.

8. Advantages:

Fast training (*<2* minutes), low memory footprint, native handling of missing values, superior PR-AUC on tabular data.

9. Limitations:

Susceptible to overfitting on noisy minority samples if `num_leaves` is too large; requires careful tree regularization.

10. Examiner / Interviewer Questions:

Why leaf-wise instead of level-wise? Leaf-wise splits the leaf with maximum loss reduction, achieving lower error with fewer total nodes than level-wise symmetric growth.

4.2 Ten-Point Teaching: SMOTE (Synthetic Minority Over-sampling)

1. What is it?

SMOTE is an imbalanced learning technique that generates synthetic minority class examples by linear interpolation between feature vectors of nearest neighbors.

2. Why is it needed?

When positive fraud instances are outnumbered 27:1, gradient boosting trees may fail to find split points separating rare fraud clusters from majority instances.

3. How does it work in general?

For each minority instance *x_i*, find its *k*-nearest minority neighbors (*k=5*). Select a random neighbor *x_{zi}* and synthesize *x_{\text{new}} = x_i + \lambda (x_{zi} - x_i)* where *\lambda \sim U(0,1)*.

4. How is it implemented in FraudLens?

In `src/train_advanced.py` using `imblearn.over_sampling.SMOTE(sampling_strategy=0.1, random_state=42)`.

5. Which file implements it?

`src/train_advanced.py`.

6. Step-by-Step Execution:

Chronological train split is isolated \to SMOTE fits to train \to minority class count is increased from *\approx 14,000* to *\approx 40,000* (1:10 ratio) \to LightGBM trains on augmented data.

7. Real-World Example:

If fraud examples exist at amounts *\350 and* \400 on card type 100, SMOTE generates synthetic frauds at \$375 on card type 100, densifying the fraud decision region.

8. Advantages:

Expands minority class boundary without exact duplicates (unlike random oversampling which causes overfitting).

9. Limitations:

Can generate unrealistic synthetic samples in high-dimensional sparse spaces; slightly causes the model to overstate probabilities (evidenced by our calibration analysis).

10. Examiner / Interviewer Questions:

Did SMOTE cause data leakage in your validation set? No, SMOTE was strictly applied to the training split *after* temporal partitioning; validation and test splits remained 100% natural and untouched.

4.3 Ten-Point Teaching: TreeSHAP Explainability

1. What is it?

TreeSHAP is an explainable AI algorithm that computes exact Shapley values from cooperative game theory for tree-based ensemble models in polynomial time.

2. Why is it needed?

Black-box predictions cannot be defended in banking audits. Compliance regulations (e.g. FCRA, GDPR) mandate adverse action notices explaining why an alert was raised.

3. How does it work in general?

It calculates the marginal contribution of each feature across all possible subsets of features, ensuring efficiency, symmetry, dummy player, and additivity axioms.

4. How is it implemented in FraudLens?

In **`api/main.py`** using **`shap.TreeExplainer(_state["model"])`**.

5. Which file implements it?

`api/main.py` (lines 89–115, 184–196).

6. Step-by-Step Execution:

X feature row *\to* **`explainer.shap_values(X)`** *\to* returns attribution vector *\phi \in \mathbb{R}^{545} \to* argsort *|\phi|* descending *\to* slice top 5 *\to* output to JSON reason codes.

7. Real-World Example:

Transaction probability is 81%. SHAP reveals **`card1_addr1_amt_mean = 144.14`** contributed *+0.32* to log-odds, while **`dist1 = 18`** contributed *-0.04*.

8. Advantages:

Exact mathematical attributions, polynomial time complexity *O(T L D^2)*, fast inference (*<10* ms).

9. Limitations:

Assumes feature independence when calculating expectations; tree structure can alter attributions if correlated features are split arbitrarily.

10. Examiner / Interviewer Questions:

Why not use LIME? LIME relies on random perturbation sampling and linear approximations which are non-deterministic, slow, and can yield unstable explanations. TreeSHAP is exact and deterministic.

4.4 Verified Model Performance Benchmarks

IEEE-CIS Benchmark Comparison (*N=88,581* Validation Split)

Model Architecture	PR-AUC	ROC-AUC	Precision @ 2% FP	Recall @ 2% FP	F1 @ Budget
Logistic Regression (Class-Weighted)	0.3050	0.8180	0.3740	0.3360	0.3540
LightGBM + scale_pos_weight	0.5520	0.9160	0.4850	0.5290	0.5060
LightGBM + SMOTE (Phase 4 Baseline)	0.5566	0.9042	0.4900	0.5410	0.5150
LightGBM + SMOTE Tuned (Phase 5, Production)	**0.5975**	**0.9242**	**0.5071**	**0.5786**	**0.5405**

Final Held-Out Test Set Generalization (IEEE-CIS, *N=88,581*)

Evaluated strictly one-shot on unseen future transactions at fixed threshold $\tau = 0.129365$:

- **Test PR-AUC: 0.53241** (Val: 0.59753, $\Delta = -0.06512$; delivering a **15.30x lift** over the 3.48% base rate).
- **Test ROC-AUC: 0.90657** (Val: 0.92419, $\Delta = -0.01762$).
- **Test Precision: 43.95%** (Val: 50.71%).
- **Test Recall: 53.39%** (Val: 57.86%).
- **Test F1 Score: 0.4821** (Val: 0.5405).
- **Actual False-Positive Rate: 2.4550%** (target $\le 2.0\%$; slight increase due to natural temporal drift).
- **Transactions Flagged for Review: 4.23%** (Val: 3.92%).
- **Probability Calibration (Brier Score):** $BS = \mathbf{0.02213}$ vs non-informative base rate $BS_{\text{base}} = \mathbf{0.03359}$, yielding a **Brier Skill Score of 34.12%**.

Chapter 5: Evidence-Grounded Agentic AI Investigation Copilot

5.1 Architecture & Workflow

```
flowchart TD
    Start([Investigate Triggered: TransactionID]) --&gt; Tools[Deterministic Evidence Assembly]

    subgraph Evidence Tools
        Tools --&gt; T1[get_entity_history: Point-in-Time card1+addr1 Stats]
        Tools --&gt; T2[get_related_entities: Distinct Domain Velocity]
        Tools --&gt; T3[Policy Query Builder: Evidence-Derived Query]
        T3 --&gt; T4[retrieve_policy_clause: TF-IDF Cosine Similarity]
    end

    T1 &amp;amp; T2 &amp;amp; T4 --&gt; CheckEntity{Valid Entity History Available?}

    CheckEntity -- No (Missing card1/addr1) --&gt; FastFallback[Fast Path: Code-Generated Insufficient Data File]
    FastFallback --&gt; FinalCase[Final Structured Case File]

    CheckEntity -- Yes --&gt; Prompt[Construct Single Evidence Prompt]
    Prompt --&gt; LLM[Call Local Ollama: Llama 3.1 8B]

    LLM --&gt; CleanCheck{Passes _model_text_clean?}

    subgraph Hardened Automated Validator
        CleanCheck -- Check 1 --&gt; V1[All numbers exist in get_entity_history / get_related_entities]
        CleanCheck -- Check 2 --&gt; V2[No numbers from policy text or notes]
        CleanCheck -- Check 3 --&gt; V3[No banned evaluative words: suspicious, concerning, etc.]
        CleanCheck -- Check 4 --&gt; V4[Comparative framing requires &gt;= 2 distinct grounded numbers]
    end

    CleanCheck -- Pass --&gt; Assemble[Assemble Full Case File]
    CleanCheck -- Fail (Attempt 1) --&gt; RetryPrompt[Append Retry Note &amp;amp; Prompt LLM Once More]
    RetryPrompt --&gt; LLM
    CleanCheck -- Fail (Attempt 2) --&gt; CodeFallback[Fallback: Generate Facts from Tool Data]

    LLM -.-&gt;|CUDA / Socket Crash| LLMFallback[LLM Unavailable: Assemble Pure Tool Case File]
    LLMFallback --&gt; FinalCase

    CodeFallback --&gt; Assemble
    Assemble --&gt; FinalCase
    FinalCase --&gt; SaveDB[(Persist to SQLite: cases.db)]
```

5.2 The 5 Sections of the Case File

1. **`ENTITY SUMMARY:`** Narrative synthesis of card+address transaction volume and spend.
2. **`TOP RISK FACTORS:`** Bulleted literal facts from evidence.
3. **`RELATED ENTITY SIGNAL:`** Deterministically computed domain velocity.
4. **`RELEVANT POLICY:`** Deterministically formatted policy clause match with threshold trigger.
5. **`RECOMMENDED ACTION:`** Deterministically computed decision:

• If prior confirmed fraud *> 0* OR any policy triggered *\to* **`"Escalate for manual review — [reasons]."`**

• Otherwise *\to* **`"Monitor, no immediate action..."`**

Chapter 6: Backend Engineering & REST API Documentation

6.1 Technology Choice & Performance

Implemented in **`api/main.py`** using **FastAPI 0.136.1** running on **Uvicorn 0.46.0**. FastAPI was chosen for:

• Asynchronous ASGI event loop handling high concurrent request volumes.

• Automatic Pydantic model validation and serialization.

• Sub-15ms scoring latency.

- Auto-generated interactive Swagger UI documentation at `http://127.0.0.1:8000/docs`.

6.2 Endpoint Summary Table

Endpoint	Method	Input Parameters	Output Model	Description
`/health`	`GET`	None	`dict`	Returns service status, model load flag, model type, and `model_source`.
`/score/{transaction_id}`	`POST`	`transaction_id: int`	`ScoreResponse`	Returns fraud risk score, boolean flag against 2% FP budget, threshold, and top-5 SHAP reason codes.
`/investigate/{transaction_id}`	`POST`	`transaction_id: int`	`InvestigateResponse`	Runs scoring + agent copilot, persists case to SQLite, and returns formatted case file.
`/cases`	`GET`	`limit: int = 20`	`list[CaseSummary]`	Lists recent investigation records from SQLite in descending ID order.
`/cases/{case_id}`	`GET`	`case_id: int`	`CaseDetail`	Retrieves full historical case file and reason codes for specified case ID.

Chapter 7: Frontend Architecture & Analyst Dashboard

7.1 Single-File React Console (`frontend/dashboard.html`)

The user interface is an analyst investigation console built in a single self-contained HTML file using React 18 and Babel loaded via CDN:

- **No npm build step:** Eliminates node_modules friction, webpack build errors, and environment mismatches.
- **Dark Theme CSS:** Designed for high-density, low-glare compliance environments (`--bg: #10141a`, `--panel: #171d26`, `--accent: #d4a24c`).
- **Interactive Controls:**
- Transaction ID input supporting Enter-key submission.
- **Get Score** button triggering instant scoring and horizontal SHAP bars (red = risk-increasing, green = risk-decreasing).
- **Investigate** button triggering the agent and rendering highlighted case sections.
- Clickable investigation sidebar history pulling from `/cases`.

Chapter 8: Database Architecture & Case Persistence

8.1 SQLite Implementation (`cases.db`)

Implemented via Python's standard `sqlite3` library. The `cases` table schema:

```
CREATE TABLE IF NOT EXISTS cases (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    transaction_id INTEGER NOT NULL,
    risk_score REAL,
    threshold_used REAL,
    flagged INTEGER,
    top_reason_codes TEXT,
    case_file TEXT,
    created_at TEXT NOT NULL
);
```

- `top_reason_codes` stores the JSON-serialized array of top-5 SHAP reason codes.
- `created_at` stores ISO-8601 UTC timestamps.

- Zero-configuration, serverless, single-file storage allows complete portability across local and evaluation environments.

Chapter 9: MLOps: Model Registry, Promotion & Population Stability Index Drift

9.1 MLflow Model Registry (`src/register_model.py`)

- Backend store: SQLite database **`mlflow.db`**.
- Registered Model Name: **`fraudlens-ieee-lightgbm`**.
- Modern Alias Assignment: **`@production`** pointer assigned to Version 3.
- Dual Loader: **`api/main.py`** dynamically queries **`models:/fraudlens-ieee-lightgbm@production`**. If registry lookup fails (e.g. inside Docker where host paths differ), it falls back seamlessly to **`models/ieee_lightgbm_tuned.joblib`**.

9.2 Population Stability Index (PSI) Drift Analysis (`src/drift_check.py`)

Evaluating 545 features between training (earliest 70%) and validation (subsequent 15%):

$$\text{PSI} = \sum_{b=1}^{10} (Q_b - P_b) \times \ln\left(\frac{Q_b}{P_b} \right)$$

- **535 Features:** Completely stable ($\text{PSI} < 0.10$).
- **2 Features:** Moderate shift ($0.10 \le \text{PSI} < 0.25$).
- **`TransactionDT`:** Exhibited $\text{PSI} = 8.28$, correctly diagnosed by our pipeline as a temporal partitioning artifact rather than behavioral drift.
- **7 Genuine Drift Candidates (1.3%):** Device/network headers (**`id_13`**, **`id_20`**, **`id_02`**) reflecting browser/OS updates.

Chapter 10: DevOps, Containerization & CI/CD Automation

10.1 Containerization Files

- **`Dockerfile`:** Built on **`python:3.13-slim`**, installs build compilers (**`gcc`**, **`g++`**, **`libgomp1`**), caches pip packages from **`requirements.txt`**, copies **`api/`** and **`src/`**, and runs Uvicorn on port 8000.
- **`docker-compose.yml`:** Mounts **`./data`**, **`./models`**, **`./mlflow.db`**, and **`./mlruns`** from host; maps **`OLLAMA_HOST=http://host.docker.internal:11434`**.
- **`.dockerignore`:** Excludes datasets, models, databases, **`.git`**, and tests from image context.

10.2 GitHub Actions Pipeline (`.github/workflows/ci.yml`)

Triggers on every push and PR to **`main`**:

1. Checks out repository and sets up Python 3.13.
2. Installs dependencies from **`requirements.txt`** + **`pytest`**.
3. Runs **`python -m compileall src/ api/ tests/`** for AST bytecode syntax verification.
4. Validates HTML parsing of **`frontend/dashboard.html`**.
5. Executes automated pytest suite (51 tests).
6. Executes **`docker build -t fraudlens-api:latest .`** for container reproducibility.

Chapter 11: Testing, Quality Assurance & Verification Suite

FraudLens enforces 100% test pass rates across **51 automated tests** (**`pytest tests/ -v`**):

```
tests/test_agent_logic.py: 19 passed
tests/test_api.py:         12 passed
tests/test_drift.py:        6 passed
tests/test_model.py:       14 passed
====================== 51 passed, 2 warnings in 10.78s ======================
```

- **`test_agent_logic.py` (19 tests):** Point-in-time boundary checks, label leakage prevention, numerical grounding in evidence tools, comparative framing rejection, policy matching, and Ollama failure resilience.
- **`test_api.py` (12 tests):** `/health`, `/score`, `/investigate`, `/cases`, HTTP status codes (200, 404, 503), and SQLite persistence.
- **`test_drift.py` (6 tests):** PSI computation across identical distributions, deliberate large shifts, near-constant series, and NaN values.
- **`test_model.py` (14 tests):** Model artifact presence, 545-feature input compatibility, probability distribution axioms, and TreeSHAP top-5 indexing.

Chapter 12: Technology Stack & Dependency Inventory

Package / Technology	Verified Version	Role in FraudLens
Python	3.13.9	Core programming language runtime.
pandas	3.0.0	Tabular data manipulation, time-series splitting, feature aggregations.
numpy	2.2.6	Vectorized numerical operations and quantile binning.
scikit-learn	1.8.0	Machine learning metrics, Logistic Regression baseline, TF-IDF vectorizer.
scipy	1.17.1	Statistical calculations and distribution analysis.
lightgbm	4.7.0	Gradient-boosted decision trees, histogram binning, predictive scoring.
imbalanced-learn	0.14.2	SMOTE minority class oversampling.
mlflow	3.16.0	Experiment tracking, model registry, alias-based model promotion.
shap	0.52.0	TreeSHAP exact game-theoretic feature attribution.
joblib	1.5.3	High-performance model serialization and fallback loading.
pyarrow	25.0.1	High-throughput columnar Parquet storage reading.
matplotlib	3.10.7	Generation of drift bar charts and calibration curves.
fastapi	0.136.1	Asynchronous REST serving backend.
uvicorn	0.46.0	Lightning-fast ASGI web server.
ollama	0.6.2	Local Python client for Llama 3.1 8B LLM inference.
pytest	9.1.1	Automated test runner.
reportlab	5.1.0	Professional PDF document generation.

Chapter 13: Complete End-to-End Execution Guide

1. Prerequisites (Windows PowerShell)

- Python 3.13 installed at: **`C:\Users\DHANRAJ\AppData\Local\Programs\Python\Python313\python.exe`**
- Ollama installed and running with **`llama3.1:8b`** pulled.

2. Run Test Suite

```
C:\Users\DHANRAJ\AppData\Local\Programs\Python\Python313\python.exe -m pytest tests/ -v
```

3. Start Backend API

```
C:\Users\DHANRAJ\AppData\Local\Programs\Python\Python313\python.exe -m uvicorn api.main:app --host 127.0.0.1 --port 8000 --reload
```

4. Open Analyst Dashboard

Double-click **`frontend/dashboard.html`** or open in browser. Enter Transaction ID **`3141064`**, press **Enter** to score, and click **Investigate** to generate the case file.

Chapter 14: File-by-File Codebase Walkthrough

- **`api/main.py`:** FastAPI backend application defining startup lifespan, dual model loading, endpoints (**`/health`**, **`/score`**, **`/investigate`**, **`/cases`**), and SQLite database operations.
- **`src/agent.py`:** Agentic investigation copilot. Defines point-in-time evidence tools (**`get_entity_history`**, **`get_related_entities`**), TF-IDF policy retrieval, prompt templates, hardened validator (**`_model_text_clean`**), and fallback generators.
- **`src/features.py`:** Feature engineering pipeline. Transforms raw CSVs into 576 engineered features across leakage-safe chronological splits.
- **`src/train_baseline.py`:** Baseline training script benchmarking Logistic Regression, LightGBM, and Isolation Forest across IEEE-CIS and PaySim.
- **`src/train_advanced.py`:** Advanced modeling script executing SMOTE resampling and hyperparameter optimization for tuned LightGBM.
- **`src/register_model.py`:** MLOps registry script registering tuned model in MLflow and promoting Version 3 to alias **`@production`**.
- **`src/drift_check.py`:** Population Stability Index calculation script comparing train vs. validation splits across all 545 features.
- **`src/evaluate_test.py`:** One-shot held-out test evaluation script computing PR-AUC, ROC-AUC, 2% FP budget metrics, and Brier calibration.
- **`frontend/dashboard.html`:** Single-file React 18 analyst console with dark theme, real-time scoring, SHAP reason bars, and investigation viewer.
- **`tests/test_agent_logic.py`:** Unit tests for agent temporal gating, numerical grounding, comparison rejection, and failure fallbacks.
- **`tests/test_api.py`:** Integration tests for FastAPI endpoints, status codes, and SQLite persistence.
- **`tests/test_drift.py`:** Unit tests for PSI mathematical computation.
- **`tests/test_model.py`:** Unit tests for model serialization, feature dimensions, probability axioms, and TreeSHAP integration.

Chapter 15: Technical Decision Justifications & Architecture Trade-Offs

- **Why LightGBM instead of XGBoost?** LightGBM trains 5x faster with 70% lower memory usage via histogram binning, crucial for 500K+ transaction datasets.
- **Why SMOTE instead of simple random oversampling?** Random oversampling duplicates exact minority instances, causing tree models to memorize specific noise points. SMOTE creates synthetic interpolations, broadening the decision boundary.
- **Why Chronological Splitting instead of Random K-Fold?** Random splits leak future transactions from the same card/entity into the training set, artificially inflating validation performance.
- **Why TreeSHAP instead of LIME?** LIME relies on random perturbation sampling and is slow and heuristic. TreeSHAP computes exact Shapley values in polynomial time (*<10* ms).
- **Why Local Llama 3.1 8B via Ollama instead of OpenAI GPT-4?** Banking regulations strictly prohibit transmitting customer PII and transaction records to public cloud APIs; local Ollama inference ensures 100% data privacy and zero recurring token costs.
- **Why SQLite instead of PostgreSQL?** SQLite is serverless, zero-configuration, self-contained in a single file, and avoids external service dependencies during academic evaluations.

- **Why React via CDN instead of a Node.js/Webpack build?** Eliminates heavy `node_modules` folders, npm dependency vulnerabilities, and complex build scripts; runs instantly in any browser.

Chapter 16: System Limitations, Risks & Realistic Future Scope

1. **Dataset Storage vs. Live Streaming:** In this project, transactions are looked up by ID from processed Parquet files. A production deployment would ingest streaming JSON events via Apache Kafka and maintain running entity counters in Redis.
2. **Local LLM Memory Requirements:** Running Llama 3.1 8B requires *\ge 6* GB of RAM, preventing the full LLM agent from running on free-tier cloud virtual machines.
3. **Database Concurrency:** SQLite supports single-writer concurrency. Production deployment with multiple concurrent human analysts requires migrating to PostgreSQL.
4. **Temporal Concept Drift:** Our held-out test evaluation showed an empirical false-positive rate of 2.46% (slightly above the 2.0% budget), proving that continuous drift monitoring and automated retraining pipelines are essential.

Chapter 17: Complete Presentation, Pitch & Demonstration Guide

(Refer to [`docs/FRAUDLENS_PRESENTATION_AND_VIVA_GUIDE.md`](FRAUDLENS_PRESENTATION_AND_VIVA_GUIDE.md) for full spoken presentation scripts, timed 5-minute academic walk-throughs, and live demonstration choreography.)

Chapter 18: Viva Voce, Technical Interview & Industry Question Bank (100+ Questions)

(Refer to [`docs/FRAUDLENS_PRESENTATION_AND_VIVA_GUIDE.md`](FRAUDLENS_PRESENTATION_AND_VIVA_GUIDE.md) Part 2 for the complete catalog of 100+ technical questions and detailed answers across Categories A through Q.)

Chapter 19: Comprehensive Glossary of Technical Terms

- **Class Imbalance:** Disproportionate ratio between majority (legitimate) and minority (fraudulent) instances.
- **PR-AUC (Precision-Recall Area Under Curve):** The definitive evaluation metric for imbalanced fraud classification, measuring precision trade-offs across all recall thresholds.
- **ROC-AUC:** Area under the True Positive Rate vs False Positive Rate curve. Measures global ranking quality.
- **Brier Score:** Mean squared error of probabilistic predictions; measures probability calibration.
- **TreeSHAP:** Algorithm computing exact game-theoretic Shapley value attributions for tree ensembles in polynomial time.
- **Population Stability Index (PSI):** Statistical metric measuring divergence between reference and target distributions. *\text{PSI} < 0.10* denotes stability.
- **SMOTE:** Synthetic Minority Over-sampling Technique; interpolates synthetic feature vectors between minority nearest neighbors.
- **Lookahead Leakage:** Contamination of training data with future information, resulting in unrealistically high validation performance.
- **Tool Calling:** Mechanism allowing an LLM agent to invoke external executable code to query data or execute actions.
- **Deterministic Logic:** Code that consistently produces identical outputs for identical inputs, without stochastic variation.

Chapter 20: Self-Assessment Knowledge & Readiness Checklist

Use this 5-point self-assessment scale:

1 = Do not understand | 2 = Recognize concept | 3 = Can explain general concept | 4 = Can explain FraudLens implementation | 5 = Can defend technical decision against an examiner.

- [ ] **Problem Formulation:** Why accuracy fails; the asymmetric cost matrix (*C_{\text{FN}} \gg C_{\text{FP}}*). (Target: 5)

- [ ] **Data Partitioning:** Why chronological 70/15/15 splitting is mandatory to prevent lookahead leakage. (Target: 5)
- [ ] **Feature Engineering:** Why `(card1, addr1)` entity aggregations were built and how they rank in top 7% of SHAP. (Target: 5)
- [ ] **The Anomaly Detection Fallacy:** Why supervised LightGBM outperformed Isolation Forest on PaySim (94.5% vs 2.5% recall). (Target: 5)
- [ ] **Gradient Boosting & LightGBM:** Histogram binning, leaf-wise growth, `-999` missing value routing. (Target: 5)
- [ ] **SMOTE:** Linear interpolation mathematics and `sampling_strategy = 0.1`. (Target: 5)
- [ ] **Evaluation Metrics:** PR-AUC vs ROC-AUC; Brier Skill Score (34.12%); the 2% false-positive budget threshold ($\tau = 0.129365$). (Target: 5)
- [ ] **TreeSHAP:** Shapley value efficiency, symmetry, dummy player, and additivity axioms; local reason code calculation. (Target: 5)
- [ ] **Agentic AI Architecture:** The 4 hallucination classes; why code decides and LLMs narrate; numerical validator and comparative framing regex. (Target: 5)
- [ ] **Backend & Serving:** FastAPI lifespan startup, dual model loader (MLflow alias vs Joblib fallback), CORS, and SQLite schema. (Target: 5)
- [ ] **MLOps & Drift:** MLflow Model Registry `@production` alias; PSI mathematical formula; the `TransactionDT` artifact diagnosis. (Target: 5)
- [ ] **DevOps & Testing:** Dockerfile multi-stage layers; GitHub Actions CI pipeline; 51 pytest test cases. (Target: 5)